

TP1



Universidade do Minho
Escola de Engenharia

Redes de Computadores

RC-TP1-PL52

Hasti Abdollahvand - A102185
Sívio João Freitas Pontes - A100762

Índice

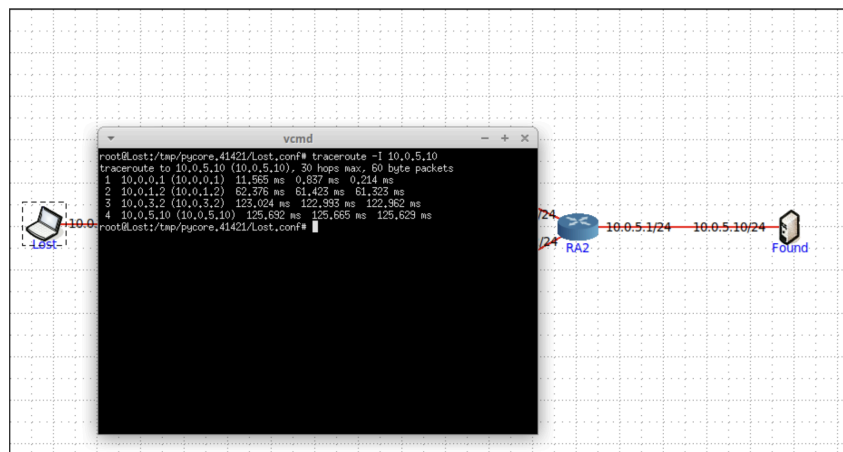
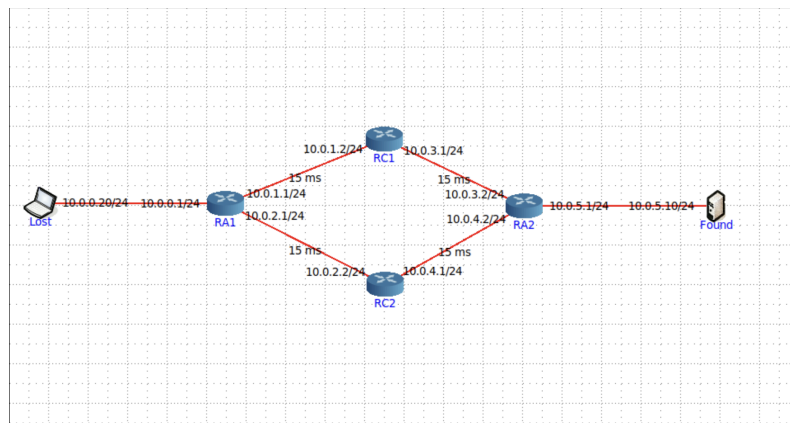
Pergunta 1	3
a)	3
b)	4
c)	5
Pergunta 2	5
a)	6
b)	6
c)	6
d)	6
e)	6
f)	7
g)	7
i)	7
ii)	8
Pergunta 3	8
a)	8
b)	10
c)	11
d)	13
e)	14
f)	15
g)	16
h)	16
i)	17
Pergunta 1	17
a)	18
b)	19
c)	21
Pergunta 2	23
a)	23
b)	24
c)	25
d)	28
i)	29
ii)	31
e)	31
f)	33
g)	34
Pergunta 3	34
Conclusão	35

Parte 1

Pergunta 1

a)

Active o Wireshark no host Lost. Numa shell de Lost execute o comando `traceroute -I` para o endereço IP do Found. Registre e analise o tráfego ICMP enviado pelo sistema Lost e o tráfego ICMP recebido como resposta. Explique os resultados obtidos tendo em conta o princípio de funcionamento do traceroute.



No.	Time	Source	Destination	Protocol	Length	Info
54	73.278507359	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0018, seq=1/256, ttl=1 (no response..
55	73.287537448	10.0.0.1	10.0.0.20	ICMP	102	Time-to-live exceeded (Time to live exceeded in transit)
56	73.292975503	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0018, seq=2/512, ttl=1 (no response..
57	73.29597362	10.0.0.1	10.0.0.20	ICMP	102	Time-to-live exceeded (Time to live exceeded in transit)
58	73.293886855	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0018, seq=3/768, ttl=1 (no response..
59	73.294017522	10.0.0.1	10.0.0.20	ICMP	102	Time-to-live exceeded (Time to live exceeded in transit)
60	73.294156086	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0018, seq=4/1024, ttl=2 (no respons..
61	73.295114576	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0018, seq=5/1280, ttl=2 (no respons..
62	73.295225203	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0018, seq=6/1536, ttl=2 (no respons..
63	73.295287501	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0018, seq=7/1792, ttl=3 (no respons..
64	73.295337786	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0018, seq=8/2048, ttl=3 (no respons..
65	73.295387792	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0018, seq=9/2304, ttl=3 (no respons..
66	73.295442267	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0018, seq=10/2560, ttl=4 (reply in ..
67	73.295493949	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0018, seq=11/2816, ttl=4 (reply in ..
68	73.295543675	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0018, seq=12/3072, ttl=4 (reply in ..
69	73.295609605	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0018, seq=13/3328, ttl=5 (reply in ..
70	73.295674975	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0018, seq=14/3584, ttl=5 (reply in ..
71	73.295727775	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0018, seq=15/3840, ttl=5 (reply in ..
72	73.295782809	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0018, seq=16/4096, ttl=6 (reply in ..
73	73.305638970	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0018, seq=17/4352, ttl=6 (reply in ..
74	73.305758816	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0018, seq=18/4608, ttl=6 (reply in ..
75	73.305844580	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0018, seq=19/4864, ttl=7 (reply in ..
76	73.356447046	10.0.1.2	10.0.0.20	ICMP	102	Time-to-live exceeded (Time to live exceeded in transit)
77	73.356494538	10.0.1.2	10.0.0.20	ICMP	102	Time-to-live exceeded (Time to live exceeded in transit)
78	73.356511579	10.0.1.2	10.0.0.20	ICMP	102	Time-to-live exceeded (Time to live exceeded in transit)
79	73.360526556	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0018, seq=20/5120, ttl=7 (reply in ..
80	73.360901748	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0018, seq=21/5376, ttl=7 (reply in ..
81	73.361129708	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0018, seq=22/5632, ttl=8 (reply in ..
82	73.418248844	10.0.3.2	10.0.0.20	ICMP	102	Time-to-live exceeded (Time to live exceeded in transit)
83	73.418299688	10.0.3.2	10.0.0.20	ICMP	102	Time-to-live exceeded (Time to live exceeded in transit)
84	73.418317847	10.0.3.2	10.0.0.20	ICMP	102	Time-to-live exceeded (Time to live exceeded in transit)
85	73.421074588	10.0.5.10	10.0.0.20	ICMP	74	Echo (ping) reply id=0x0018, seq=10/2560, ttl=61 (request ..
86	73.421125991	10.0.5.10	10.0.0.20	ICMP	74	Echo (ping) reply id=0x0018, seq=11/2816, ttl=61 (request ..
87	73.421143311	10.0.5.10	10.0.0.20	ICMP	74	Echo (ping) reply id=0x0018, seq=12/3072, ttl=61 (request ..

Podemos ver o traceroute a aumentar o ttl para mapear o a rede, vai recebendo mensagens de ICMP como ttl (time to live) foi excedido e e vai aumentando o ttl por 1. Podemos observar que o traceroute manda rajadas de requests para garantir uma resposta ao ping. Assim segue a rota RA1-RC1-RA2-Found.

b)

Qual deve ser o valor inicial mínimo do campo TTL para alcançar o servidor Found?
Esboce um esquema com o valor do campo TTL à chegada a cada um dos routers percorridos até ao servidor Found. Verifique na prática que a sua resposta está correta.

O TTL teria que ser 4. Podemos observar na print acima que recebemos TTL exceeded com TTL 1, 2 e 3. No entanto qualquer número superior foi recebida um reply.

10.0.0.1 - 10.0.1.2 - 10.0.3.1 - 10.0.5.10
 RA1 - RC1 - RA2 - Found

```
root@Lost:/tmp/pycore.41421/Lost.conf# ping 10.0.5.10 -t 4
PING 10.0.5.10 (10.0.5.10) 56(84) bytes of data.
64 bytes from 10.0.5.10: icmp_seq=1 ttl=61 time=62.0 ms
64 bytes from 10.0.5.10: icmp_seq=2 ttl=61 time=61.9 ms
64 bytes from 10.0.5.10: icmp_seq=3 ttl=61 time=62.2 ms
64 bytes from 10.0.5.10: icmp_seq=4 ttl=61 time=62.2 ms
64 bytes from 10.0.5.10: icmp_seq=5 ttl=61 time=62.3 ms
^C
--- 10.0.5.10 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4007ms
rtt min/avg/max/mdev = 61.904/62.131/62.313/0.149 ms
root@Lost:/tmp/pycore.41421/Lost.conf#
```

Podemos observar que o ping com o ttl=4 recebe um reply. (O ttl que aparece na lista é o do reply).

c)

Calcule o valor médio do tempo de ida-e-volta (RTT - Round-Trip Time) obtido no acesso ao servidor. Por modo a obter uma média mais confiável, poderá alterar o número pacotes de prova com a opção -q.

```
root@Lost:/tmp/pycore.41421/Lost.conf# traceroute -I -q 2 10.0.5.10
traceroute to 10.0.5.10 (10.0.5.10), 30 hops max, 60 byte packets
 1  10.0.0.1 (10.0.0.1)  4.039 ms  0.331 ms
 2  10.0.1.2 (10.0.1.2)  31.559 ms  31.275 ms
 3  10.0.3.2 (10.0.3.2)  61.637 ms  61.527 ms
 4  10.0.5.10 (10.0.5.10)  62.698 ms  62.597 ms
```

O RTT (Round-Trip Time) corresponde ao tempo de ida e volta de um pacote entre a origem e o destino, sendo medido diretamente através do comando traceroute. Os valores apresentados já representam o RTT, não sendo necessário duplicar o tempo de ida. Para obter uma média mais fiável, foi utilizado o comando com a opção -q, mantendo-se valores consistentes.

Pergunta 2

No	Time	Source	Destination	Protocol	Length	Time
58	1.481523	172.26.77.180	193.136.9.240	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=617) (Reassembled in #52)
51	1.481548	172.26.77.180	193.136.9.240	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=1488, ID=617) (Reassembled in #52)
52	1.481547	172.26.77.180	193.136.9.240	ICMP	666	Echo (ping) request id=0xa616, seq=1/256, ttl=1 (no response found)
53	1.481562	172.26.77.180	193.136.9.240	ICMP	70	Time-to-live exceeded (time to live exceeded in transit)
54	1.480839	172.26.77.180	193.136.9.240	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=618) (Reassembled in #56)
55	1.480869	172.26.77.180	193.136.9.240	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=1488, ID=618) (Reassembled in #56)
56	1.480874	172.26.77.180	193.136.9.240	ICMP	666	Echo (ping) request id=0xa616, seq=2/256, ttl=1 (no response found)
57	1.481892	172.26.77.180	193.136.9.240	ICMP	70	Time-to-live exceeded (time to live exceeded in transit)
58	1.781362	172.26.77.180	193.136.9.240	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=619) (Reassembled in #60)
59	1.781383	172.26.77.180	193.136.9.240	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=1488, ID=619) (Reassembled in #60)
60	1.781388	172.26.77.180	193.136.9.240	ICMP	666	Echo (ping) request id=0xa616, seq=3/256, ttl=1 (no response found)
61	1.785825	172.26.254.254	172.26.77.180	ICMP	70	Time-to-live exceeded (time to live exceeded in transit)
62	1.785187	172.26.77.180	193.136.9.240	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=620) (Reassembled in #64)
63	1.785158	172.26.77.180	193.136.9.240	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=1488, ID=620) (Reassembled in #64)
64	1.785155	172.26.77.180	193.136.9.240	ICMP	666	Echo (ping) request id=0xa616, seq=4/256, ttl=2 (no response found)
65	1.785824	172.26.77.180	193.136.9.240	ICMP	70	Time-to-live exceeded (time to live exceeded in transit)
66	1.718847	172.26.77.180	193.137.16.65	DNS	84	Standard query 0x5f2 PTR 12.1.16.172.in-addr.arpa
67	1.718852	172.26.77.180	193.137.16.75	DNS	84	Standard query 0x5f2 PTR 12.1.16.172.in-addr.arpa
68	1.718218	172.26.77.180	193.137.16.145	DNS	84	Standard query 0x5f2 PTR 12.1.16.172.in-addr.arpa
69	1.713585	193.137.16.65	172.26.77.180	DNS	84	Standard query response 0x5f2 Refused PTR 12.1.16.172.in-addr.arpa

> Frame 10: Packet, 1514 bytes on wire (12112 bits), 1514 bytes captured (12112 bits) on interface
> Ethernet II, Src: Apple-Sai74:10 (1c:86:38:5a:74:10), Dst: Cisco-16:e2:9f (d4:7f:35:16:e2:9f)
> Internet Protocol Version 4, Src: 172.26.77.180, Dst: 193.136.9.240
> ... 0101 = Version: 4
> ... 0101 = Header Length: 20 bytes (5)
> Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
> Total Length: 1508
> Identification: 0xa617 (42519)
> 001. = Flags: 0x1: More fragments
> 0. = Reserved bit: Not set
> 0. = Don't fragment: Not set
> 1. = More fragments: Set
> 0000 0000 0000 = Fragment Offset: 0
> Time to Live: 1
> Protocol: ICMP (1)
> Header checksum: 0x2933 (validation disabled)
> [Header checksum status: Unverified]
> Source Address: 172.26.77.180
> Destination Address: 193.136.9.240
> Fragment Offset (to frag_offset), 13 bits
> Packets: 130 - Dropped: 0 (0.0%)
> Profile: Default

```

~ > traceroute -I marco.uminho.pt 3612 17:39:48
traceroute to marco.uminho.pt (193.136.9.240), 64 hops max, 3612 byte packets
 1 172.26.254.254 (172.26.254.254) 9.752 ms 29.076 ms 21.865 ms
 2 172.16.1.12 (172.16.1.12) 14.219 ms 6.836 ms 94.203 ms
 3 172.16.115.252 (172.16.115.252) 11.343 ms 25.127 ms 6.077 ms
 4 marco.uminho.pt (193.136.9.240) 6.462 ms 50.420 ms 15.762 ms
~ > 17:39:50

```

a)

Qual é o endereço IP da interface ativa do seu computador?

172.26.77.100

```

Protocol: ICMP (1)
Header Checksum: 0x481e [validation disabled]
[Header checksum status: Unverified]
Source Address: 172.26.77.100
Destination Address: 193.136.9.240
[3 IPv4 Fragments (3592 bytes): #189(1480), #190(1480), #191(632)]
[Stream index: 21]

```

b)

Qual é o valor do campo protocol?
O que permite Identificar?

ICMP (1)

c)

Quantos bytes tem o cabeçalho IPv4? Quantos bytes tem o campo de dados (payload) do datagrama? Como se calcula o tamanho do payload?

O header têm 20 Bytes e o tamanho total é 1500 bytes. Ou seja o payload é 1480 bytes. Temos vários packets devido a fragmentação, o ultimo sendo o unico diferente que têm 632bytes + 20 Bytes de header também

```

.... 0101 = Header Length: 20 bytes (5)
> Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
Total Length: 1500

```

d)

O datagrama IP foi fragmentado?
Justifique.

Sim foi fragmentado, conseguimos ver que a fragflag está a 1.

```

v 001. .... = Flags: 0x1, More fragments
    0... .... = Reserved bit: Not set
    .0.. .... = Don't fragment: Not set
    ..1. .... = More fragments: Set
    ...0 0000 1011 1001 = Fragment Offset: 1480
> Time to Live: 4

```

e)

Analise a sequência de tráfego ICMP gerado a partir do endereço IP atribuído à interface da sua máquina. Para a sequência de mensagens ICMP enviadas pelo seu computador,

indique que campos do cabeçalho IP variam de pacote para pacote. Justifique estas mudanças.

TTL, ID e Header checksum. O payload, offset e fragflag também mudam mediante o fragmento. TTL incrementam 1 a 1, ID muda mediante o packet, se for fragmentado mantém o ID e o header checksum muda pois o cabeçalho muda

f)

Observa algum padrão nos valores do campo de Identificação do datagrama IP e TTL?

No funcionamento do traceroute, o valor inicial do TTL (Time To Live) é incrementado progressivamente em cada conjunto de pacotes enviados, permitindo descobrir os routers ao longo do caminho. No entanto, durante o encaminhamento, o valor do TTL é decrementado em cada salto efetuado, sendo este o comportamento normal do protocolo IP.

O campo Identification varia entre datagramas IP e é utilizado para identificar fragmentos pertencentes ao mesmo pacote original. Embora em alguns sistemas possa apresentar um padrão sequencial, tal comportamento não é garantido, dependendo da implementação do sistema operativo.

g)

Ordene o tráfego capturado por endereço destino e encontre a série de respostas ICMP TTL Exceeded enviadas ao seu computador.

```
178 6.108022 172.16.115.252 172.26.77.100 ICMP 70 Time-to-live exceeded (Time to live exceeded in transit)
182 6.884925 172.16.115.252 172.26.77.100 ICMP 70 Time-to-live exceeded (Time to live exceeded in transit)
186 6.811357 193.136.9.240 172.26.77.100 IPv4 1514 Fragmented IP protocol (proto=ICMP 1, off=0, ID=268) (Reassembled in #188)
187 6.811362 193.136.9.240 172.26.77.100 IPv4 666 Fragmented IP protocol (proto=ICMP 1, off=256, ID=268) (Reassembled in #188)
188 6.811364 193.136.9.240 172.26.77.100 ICMP 1514 Echo (ping) reply id=654df, seq=10/2560, ttl=61 (request in 185)
192 6.835774 193.136.9.240 172.26.77.100 IPv4 1514 Fragmented IP protocol (proto=ICMP 1, off=0, ID=2689) (Reassembled in #194)
193 6.835774 193.136.9.240 172.26.77.100 IPv4 1514 Fragmented IP protocol (proto=ICMP 1, off=256, ID=2689) (Reassembled in #194)
> Frame 182: Packet, 70 bytes on wire (560 bits), 70 bytes captured (560 bits) on interface en 0000 3c 06 30 5a 74 10 d4 7f 35 16 e2 9f 00 00 45 c0 <...
> Ethernet II, Src: Cisco_16:e2:9f (d4:7f:35:16:e2:9f), Dst: Apple_Sa:74:10 (3c:06:30:5a:74:10) 0010 00 30 7f 55 00 00 f0 01 24 24 ac 10 73 fc ac 1a 0
0020 4d 64 00 00 f0 ff 00 00 00 00 45 00 05 dc 05 00 00
0030 20 00 01 01 29 42 ac 1a 4d 64 c1 88 09 f0 00 00
0040 52 17 a5 df 00 09
0100 .... = Version: 4
... 0101 = Header Length: 20 bytes (5)
> Differentiated Services Field: 0xc0 (DSCP: CS6, ECN: Not-ECT)
Total Length: 56
Identification: 0x7f55 (32597)
0000 .... = Flags: 0x0
... 0 0000 0000 0000 = Fragment Offset: 0
Time to Live: 253
Protocol: ICMP (1)
Header Checksum: 0x2424 (validation disabled)
[Header checksum status: Unverified]
Source Address: 172.16.115.252
Destination Address: 172.26.77.100
[Stream index: 25]
Internet Control Message Protocol
Type: Time-to-live exceeded (11)
Code: 0 (Time to live exceeded in transit)
```

```
158 6.621946 172.26.254.254 172.26.77.100 ICMP 70 Time-to-live exceeded (Time to live exceeded in transit)
151 6.621949 146.75.89.91 172.26.77.100 TLV... 94 Application Data
152 6.630309 172.26.254.254 172.26.77.100 ICMP 13 Time-to-live exceeded (Time to live exceeded in transit)
168 6.658158 172.16.1.12 172.26.77.100 ICMP 70 Time-to-live exceeded (Time to live exceeded in transit)
164 6.666477 172.16.1.12 172.26.77.100 ICMP 70 Time-to-live exceeded (Time to live exceeded in transit)
164 6.666477 172.16.1.12 172.26.77.100 UDP 68 413 => 6493 [len=52]
178 6.768689 172.16.1.12 172.26.77.100 ICMP 70 Time-to-live exceeded (Time to live exceeded in transit)
174 6.771069 172.16.115.252 172.26.77.100 ICMP 70 Time-to-live exceeded (Time to live exceeded in transit)
178 6.798622 172.16.115.252 172.26.77.100 ICMP 70 Time-to-live exceeded (Time to live exceeded in transit)
182 6.884925 172.16.115.252 172.26.77.100 ICMP 70 Time-to-live exceeded (Time to live exceeded in transit)
186 6.811357 193.136.9.240 172.26.77.100 IPv4 1514 Fragmented IP protocol (proto=ICMP 1, off=0, ID=268) (Reassembled in #188)
187 6.811362 193.136.9.240 172.26.77.100 IPv4 666 Fragmented IP protocol (proto=ICMP 1, off=256, ID=268) (Reassembled in #188)
188 6.811364 193.136.9.240 172.26.77.100 ICMP 1514 Echo (ping) reply id=654df, seq=10/2560, ttl=61 (request in 185)
192 6.835774 193.136.9.240 172.26.77.100 IPv4 1514 Fragmented IP protocol (proto=ICMP 1, off=0, ID=2689) (Reassembled in #194)
193 6.835774 193.136.9.240 172.26.77.100 IPv4 1514 Fragmented IP protocol (proto=ICMP 1, off=256, ID=2689) (Reassembled in #194)
> Frame 158: Packet, 70 bytes on wire (560 bits), 70 bytes captured (560 bits) on interface en 0000 3c 06 30 5a 74 10 d4 7f 35 16 e2 9f 00 00 45 c0 <...
> Ethernet II, Src: Cisco_16:e2:9f (d4:7f:35:16:e2:9f), Dst: Apple_Sa:74:10 (3c:06:30:5a:74:10) 0010 00 30 7f 55 00 00 f0 01 24 24 ac 10 73 fc ac 1a 0
0020 4d 64 00 00 f0 ff 00 00 00 00 45 00 05 dc 05 00 00
0030 20 00 01 01 29 49 ac 1a 4d 64 c1 88 09 f0 00 00
0040 52 1e a5 df 00 02
0100 .... = Version: 4
... 0101 = Header Length: 20 bytes (5)
> Differentiated Services Field: 0xc0 (DSCP: CS6, ECN: Not-ECT)
Total Length: 56
Identification: 0x1d00 (7664)
0000 .... = Flags: 0x0
... 0 0000 0000 0000 = Fragment Offset: 0
Time to Live: 254
Protocol: ICMP (1)
Header Checksum: 0xf97c (validation disabled)
[Header checksum status: Unverified]
Source Address: 172.26.254.254
Destination Address: 172.26.77.100
[Stream index: 22]
Internet Control Message Protocol
Type: Time-to-live exceeded (11)
Code: 0 (Time to live exceeded in transit)
Unexpected end of filter expression.
```

i)

Qual é o valor do campo TTL recebido no seu computador? Esse valor permanece constante para todas as mensagens de resposta ICMP TTL Exceeded recebidas no seu computador? Porquê?

O valor do TTL recebido varia, não é constante, temos packets recebidos com 253 e com 254. Isto acontece porque isto é uma nova mensagem que é enviada como resposta, independente do TTL da mensagem recebida no router. Isto pode variar do router inicial e também depende do número de saltos no regresso.

ii)

Por que razão as mensagens de resposta ICMP TTL Exceeded são sempre enviadas na origem com um valor relativamente alto?

Porque são mensagens de controlo importantes e precisam de ter grande probabilidade de chegar ao emissor original.

Pergunta 3

X = 52, o tamanho pedido é **(3610 + 52) = 3662 bytes**

```
[> ping -c 4 -s 3662 marco.uminho.pt
PING marco.uminho.pt (193.136.9.240): 3662 data bytes
3670 bytes from 193.136.9.240: icmp_seq=0 ttl=61 time=9.685 ms
3670 bytes from 193.136.9.240: icmp_seq=1 ttl=61 time=6.899 ms
3670 bytes from 193.136.9.240: icmp_seq=2 ttl=61 time=6.369 ms
3670 bytes from 193.136.9.240: icmp_seq=3 ttl=61 time=13.354 ms

--- marco.uminho.pt ping statistics ---
4 packets transmitted, 4 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 6.369/9.077/13.354/2.772 ms
```

a)

Localize a primeira mensagem ICMP. Porque é que houve necessidade de fragmentar o pacote inicial?

794	64.437238	172.26.66.135	193.136.9.240	ICMP	1514 Echo (ping) request	id=0x8508, seq=0/0, ttl=64 (reply in 797)
797	64.442247	193.136.9.240	172.26.66.135	ICMP	1514 Echo (ping) reply	id=0x8508, seq=0/0, ttl=61 (request in 794)

A primeira mensagem ICMP capturada no Wireshark é:

794 64.437238 172.26.66.135 193.136.9.240 ICMP 1514 Echo (ping) request
id=0x8508, seq=0/0, ttl=64

Essa mensagem corresponde a um ICMP Echo Request (ping) enviado do endereço IP 172.26.66.135 (origem) para o endereço 193.136.9.240 (destino), que corresponde ao servidor marco.uminho.pt. O campo TTL=64 indica o valor inicial do Time To Live definido pelo sistema operativo para o pacote enviado.

A fragmentação do pacote foi necessária devido ao tamanho do datagrama IP gerado pelo comando ping utilizado no experimento. Neste caso, o comando executado foi:

ping -c 4 -s 3662 marco.uminho.pt

Este comando envia pacotes ICMP com 3662 bytes de dados. No entanto, ao considerar também os cabeçalhos dos protocolos envolvidos, o tamanho total do datagrama IP torna-se maior. O pacote inclui:

- 3662 bytes de dados ICMP
- 8 bytes do cabeçalho ICMP
- 20 bytes do cabeçalho IPv4

Assim, o tamanho total do datagrama IP enviado é aproximadamente:

$3662 + 8 + 20 = 3690$ bytes

Nas redes Ethernet e Wi-Fi, o MTU (Maximum Transmission Unit) típico é de 1500 bytes, que representa o tamanho máximo de um datagrama IP que pode ser transmitido num único frame sem fragmentação.

Como o tamanho total do datagrama IP (3690 bytes) excede o MTU da rede (1500 bytes), o pacote não pode ser transmitido integralmente. Para resolver este problema, o protocolo IPv4 realiza fragmentação, dividindo o datagrama original em vários fragmentos menores que respeitam o limite imposto pelo MTU.

Na captura realizada no Wireshark, observa-se que os pacotes ICMP apresentados possuem um tamanho de aproximadamente 1514 bytes. Este valor corresponde ao tamanho do frame Ethernet capturado, que inclui:

- 1500 bytes do pacote IP
- 14 bytes do cabeçalho Ethernet

Este valor confirma que o datagrama original foi dividido em fragmentos compatíveis com o MTU da rede.

Portanto, pode concluir-se que a primeira mensagem ICMP foi fragmentada, pois o tamanho do datagrama IP gerado pelo comando ping excede o MTU da rede. A fragmentação foi necessária para permitir que os dados fossem transmitidos através da rede respeitando as restrições de tamanho impostas pelos enlaces ao longo do caminho entre o remetente e o destinatário.

No.	Time	Source	Destination	Protocol	Length	Info
315	15.974732	172.26.66.135	17.253.28.69	ICMP	70	Destination unreachable (Port unreachable)
316	15.974758	172.26.66.135	17.253.28.69	ICMP	70	Destination unreachable (Port unreachable)
318	16.056890	172.26.66.135	17.253.28.69	ICMP	70	Destination unreachable (Port unreachable)
321	16.242067	172.26.66.135	17.253.28.69	ICMP	70	Destination unreachable (Port unreachable)
322	16.242097	172.26.66.135	17.253.28.69	ICMP	70	Destination unreachable (Port unreachable)
327	16.691914	172.26.66.135	17.253.28.69	ICMP	70	Destination unreachable (Port unreachable)
328	16.691933	172.26.66.135	17.253.28.69	ICMP	70	Destination unreachable (Port unreachable)
339	17.347967	172.26.66.135	17.253.28.69	ICMP	70	Destination unreachable (Port unreachable)
340	17.348010	172.26.66.135	17.253.28.69	ICMP	70	Destination unreachable (Port unreachable)
379	18.841997	172.26.66.135	17.253.28.69	ICMP	70	Destination unreachable (Port unreachable)
380	18.842055	172.26.66.135	17.253.28.69	ICMP	70	Destination unreachable (Port unreachable)
439	21.811794	172.26.66.135	17.253.28.69	ICMP	70	Destination unreachable (Port unreachable)
440	21.811842	172.26.66.135	17.253.28.69	ICMP	70	Destination unreachable (Port unreachable)
494	27.751109	172.26.66.135	17.253.28.69	ICMP	70	Destination unreachable (Port unreachable)
495	27.751157	172.26.66.135	17.253.28.69	ICMP	70	Destination unreachable (Port unreachable)
570	39.427885	172.26.66.135	17.253.28.69	ICMP	70	Destination unreachable (Port unreachable)
571	39.427945	172.26.66.135	17.253.28.69	ICMP	70	Destination unreachable (Port unreachable)
794	64.437238	172.26.66.135	193.136.9.240	ICMP	1514	Echo (ping) request id=0x8508, seq=0/0, ttl=64 (reply in 797)
797	64.442247	193.136.9.240	172.26.66.135	ICMP	1514	Echo (ping) reply id=0x8508, seq=0/0, ttl=61 (request in 794)
814	65.442504	172.26.66.135	193.136.9.240	ICMP	1514	Echo (ping) request id=0x8508, seq=1/256, ttl=64 (reply in 817)
817	65.447163	193.136.9.240	172.26.66.135	ICMP	1514	Echo (ping) reply id=0x8508, seq=1/256, ttl=61 (request in 814)
829	66.446702	172.26.66.135	193.136.9.240	ICMP	1514	Echo (ping) request id=0x8508, seq=2/512, ttl=64 (reply in 832)
832	66.450872	193.136.9.240	172.26.66.135	ICMP	1514	Echo (ping) reply id=0x8508, seq=2/512, ttl=61 (request in 829)
871	67.448119	172.26.66.135	193.136.9.240	ICMP	1514	Echo (ping) request id=0x8508, seq=3/768, ttl=64 (reply in 874)
874	67.459219	193.136.9.240	172.26.66.135	ICMP	1514	Echo (ping) reply id=0x8508, seq=3/768, ttl=61 (request in 871)

b)

Imprima o primeiro fragmento do datagrama IP segmentado. Que informação no cabeçalho indica que o datagrama foi fragmentado? Que informação no cabeçalho IP indica que se trata do primeiro fragmento? Qual é o tamanho deste datagrama IP?

Wireshark · Packet 794 · Wi-Fi: en0

```

> Frame 794: Packet, 1514 bytes on wire (12112 bits), 1514 bytes captured (12112 bits) on interface en0, id 0
> Ethernet II, Src: Apple_aa:8c:0a (f0:18:98:aa:8c:0a), Dst: Cisco_16:e2:9f (d4:7f:35:16:e2:9f)
  > Internet Protocol Version 4, Src: 172.26.66.135, Dst: 193.136.9.240
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
      0000 00.. = Differentiated Services Codepoint: Default (0)
      .... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
    Total Length: 1500
    Identification: 0x88a6 (34982)
    > 001. .... = Flags: 0x1, More fragments
      0... .... = Reserved bit: Not set
      .0.. .... = Don't fragment: Not set
      ..1. .... = More fragments: Set
      ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: ICMP (1)
    Header Checksum: 0x1261 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 172.26.66.135
    Destination Address: 193.136.9.240
    [Stream index: 51]
  > Internet Control Message Protocol
  
```

O primeiro fragmento do datagrama IP corresponde ao pacote capturado no Wireshark com as seguintes características:

Frame 794: 1514 bytes on wire

Src: 172.26.66.135

Dst: 193.136.9.240

Protocol: ICMP

Este pacote corresponde a um ICMP Echo Request (ping) enviado do endereço IP 172.26.66.135 para o endereço 193.136.9.240.

Ao observar o cabeçalho IPv4, verifica-se que o campo Flags apresenta o valor More fragments: Set, indicando que o datagrama IP foi fragmentado e que existem mais fragmentos pertencentes ao mesmo datagrama.

Além disso, o campo Fragment Offset apresenta o valor 0, o que indica que este fragmento corresponde ao primeiro fragmento do datagrama original, uma vez que começa na posição inicial dos dados.

O campo Total Length apresenta o valor 1500 bytes, que corresponde ao tamanho total do datagrama IP neste fragmento, incluindo o cabeçalho IP e os dados transportados. Este valor está de acordo com o MTU típico das redes Ethernet, que é aproximadamente 1500 bytes, o que explica porque o datagrama original teve de ser fragmentado.

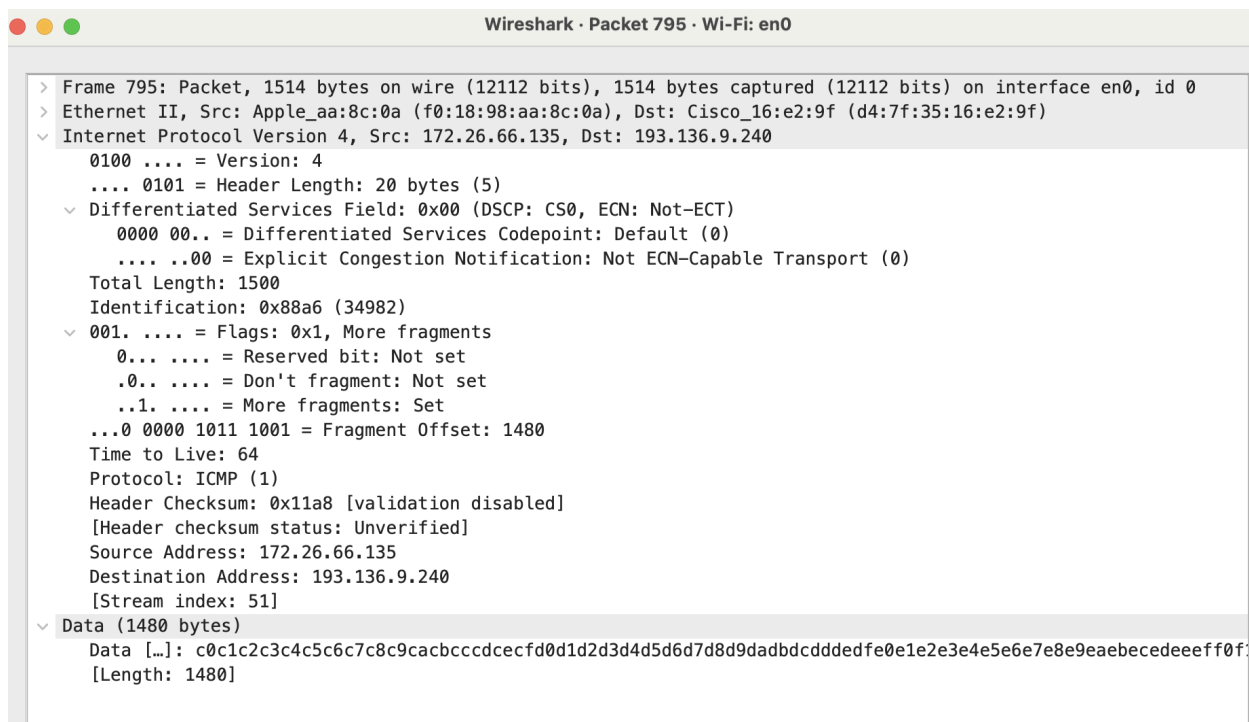
O campo Identification = 0x88a6 é utilizado para identificar todos os fragmentos que pertencem ao mesmo datagrama IP original, permitindo que o recetor consiga posteriormente reconstruir o pacote completo.

Assim, pode concluir-se que o pacote analisado corresponde ao primeiro fragmento de um datagrama IP fragmentado, identificado pelo fragment offset igual a 0 e pela presença da flag More fragments ativa.

c)

Imprima o segundo fragmento do datagrama IP original. Que informação do cabeçalho IP indica que não se trata do 1º fragmento? Existem mais fragmentos? O que nos permite afirmar isso?

ip.id == 0x88a6						
No.	Time	Source	Destination	Protocol	Length	Info
794	64.437238	172.26.66.135	193.136.9.240	ICMP	1514	Echo (ping) request id=0x8508, seq=0/0, ttl=64 (reply in 797)
795	64.437263	172.26.66.135	193.136.9.240	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=1480, ID=88a6)
796	64.437275	172.26.66.135	193.136.9.240	IPv4	744	Fragmented IP protocol (proto=ICMP 1, off=2960, ID=88a6)



```
> Frame 795: Packet, 1514 bytes on wire (12112 bits), 1514 bytes captured (12112 bits) on interface en0, id 0
> Ethernet II, Src: Apple_aa:8c:0a (f0:18:98:aa:8c:0a), Dst: Cisco_16:e2:9f (d4:7f:35:16:e2:9f)
< Internet Protocol Version 4, Src: 172.26.66.135, Dst: 193.136.9.240
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  < Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    0000 00.. = Differentiated Services Codepoint: Default (0)
    .... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
  Total Length: 1500
  Identification: 0x88a6 (34982)
  < 001. .... = Flags: 0x1, More fragments
    0... .... = Reserved bit: Not set
    .0.. .... = Don't fragment: Not set
    ..1. .... = More fragments: Set
  ...0 0000 1011 1001 = Fragment Offset: 1480
  Time to Live: 64
  Protocol: ICMP (1)
  Header Checksum: 0x11a8 [validation disabled]
  [Header checksum status: Unverified]
  Source Address: 172.26.66.135
  Destination Address: 193.136.9.240
  [Stream index: 51]
  < Data (1480 bytes)
    Data [...]: c0c1c2c3c4c5c6c7c8c9cacbcccdcecfcd0d1d2d3d4d5d6d7d8d9daddbdcdddedfe0e1e2e3e4e5e6e7e8e9eaebeceedeef0f
    [Length: 1480]
```

O segundo fragmento do datagrama IP pode ser identificado no Wireshark utilizando o filtro baseado no campo Identification, que neste caso possui o valor 0x88a6. Este campo permite localizar todos os fragmentos pertencentes ao mesmo datagrama IP original.

Ao observar o segundo fragmento capturado (Frame 795), verifica-se que o cabeçalho IPv4 apresenta as seguintes informações relevantes:

- Identification: 0x88a6
- Total Length: 1500
- Fragment Offset: 1480
- Flags: More fragments

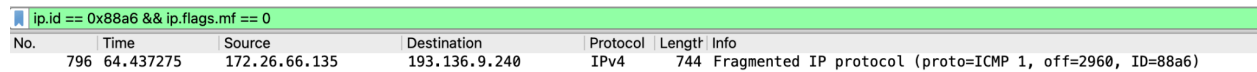
O campo Fragment Offset apresenta o valor 1480, que é diferente de zero. Este campo indica a posição dos dados deste fragmento dentro do datagrama IP original. Como o valor é superior a zero, conclui-se que este fragmento não corresponde ao primeiro fragmento, pois o primeiro fragmento teria sempre Fragment Offset = 0.

Além disso, no campo Flags verifica-se que a flag More fragments (MF) está ativa. Isto indica que existem ainda mais fragmentos pertencentes ao mesmo datagrama IP, ou seja, este fragmento não é o último fragmento.

Assim, pode concluir-se que o pacote analisado corresponde ao segundo fragmento do datagrama IP original, identificado pelo Fragment Offset diferente de zero e pela presença da flag More fragments, que indica que ainda existem fragmentos adicionais.

d)

Quantos fragmentos foram criados a partir do datagrama original? Como se detecta o último fragmento correspondente ao datagrama original? Estabeleça um filtro no Wireshark que permita listar apenas o último fragmento do primeiro datagrama IP segmentado.



No.	Time	Source	Destination	Protocol	Length	Info
796	64.437275	172.26.66.135	193.136.9.240	IPv4	744	Fragmented IP protocol (proto=ICMP 1, off=2960, ID=88a6)

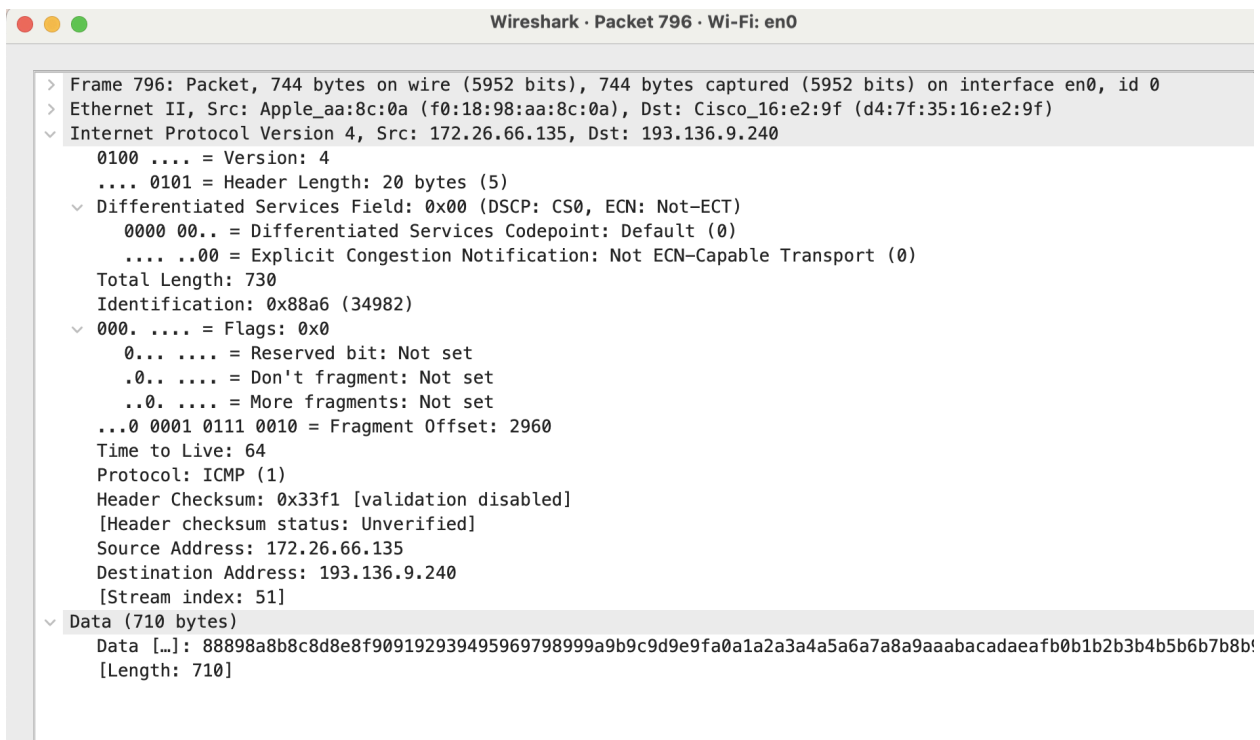
O número de fragmentos criados a partir do datagrama IP original pode ser determinado analisando os fragmentos capturados no Wireshark que possuem o mesmo valor no campo Identification. No presente caso, todos os fragmentos apresentam o valor Identification = 0x88a6, o que indica que pertencem ao mesmo datagrama IP original.

Ao observar os pacotes capturados verifica-se a existência de três fragmentos, identificados pelos diferentes valores do campo Fragment Offset:

- Primeiro fragmento: Fragment Offset = 0
- Segundo fragmento: Fragment Offset = 1480
- Terceiro fragmento: Fragment Offset = 2960

Estes valores indicam a posição de cada fragmento dentro do datagrama IP original, permitindo concluir que o pacote foi dividido em três fragmentos.

O último fragmento pode ser identificado através do campo Flags do cabeçalho IPv4. Nos fragmentos intermédios, a flag More Fragments (MF) encontra-se ativa, indicando que existem ainda fragmentos adicionais. No entanto, no último fragmento esta flag encontra-se desativada (MF = 0), o que indica que não existem mais fragmentos pertencentes ao mesmo datagrama. No Wireshark, é possível identificar diretamente o último fragmento utilizando o seguinte filtro: `ip.id == 0x88a6 && ip.flags.mf == 0`



```
> Frame 796: Packet, 744 bytes on wire (5952 bits), 744 bytes captured (5952 bits) on interface en0, id 0
> Ethernet II, Src: Apple_aa:8c:0a (f0:18:98:aa:8c:0a), Dst: Cisco_16:e2:9f (d4:7f:35:16:e2:9f)
< Internet Protocol Version 4, Src: 172.26.66.135, Dst: 193.136.9.240
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  < Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    0000 00.. = Differentiated Services Codepoint: Default (0)
    .... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
  Total Length: 730
  Identification: 0x88a6 (34982)
  < 000. .... = Flags: 0x0
    0... .... = Reserved bit: Not set
    .0.. .... = Don't fragment: Not set
    ..0. .... = More fragments: Not set
    ...0 0001 0111 0010 = Fragment Offset: 2960
  Time to Live: 64
  Protocol: ICMP (1)
  Header Checksum: 0x33f1 [validation disabled]
  [Header checksum status: Unverified]
  Source Address: 172.26.66.135
  Destination Address: 193.136.9.240
  [Stream index: 51]
  < Data (710 bytes)
    Data [...]: 88898a8b8c8d8e8f909192939495969798999a9b9c9d9e9fa0a1a2a3a4a5a6a7a8a9aaabacadaeafb0b1b2b3b4b5b6b7b8b9
    [Length: 710]
```

e)

Indique, resumindo, os campos que mudam no cabeçalho IP entre os diferentes fragmentos, e explique a forma como essa informação permite reconstruir o datagrama original.

Quando um datagrama IP é fragmentado, alguns campos do cabeçalho IPv4 mantêm-se iguais em todos os fragmentos, enquanto outros mudam entre os diferentes fragmentos. Estes campos são fundamentais para permitir que o destinatário consiga reconstruir corretamente o datagrama original.

Entre os campos que mudam entre os fragmentos, destacam-se os seguintes:

- Total Length – indica o tamanho total de cada fragmento IP (cabeçalho + dados). Como cada fragmento pode transportar quantidades diferentes de dados, este valor pode variar entre fragmentos, especialmente no último fragmento, que normalmente é menor.
- Fragment Offset – indica a posição dos dados desse fragmento dentro do datagrama original. O primeiro fragmento tem sempre offset igual a 0, enquanto os fragmentos seguintes apresentam valores maiores que indicam a posição onde os seus dados devem ser colocados no datagrama original.
- Flags (More Fragments – MF) – este campo indica se existem mais fragmentos depois do fragmento atual. Nos fragmentos intermédios a flag MF está ativa (MF = 1), enquanto no último fragmento MF = 0, indicando que não existem mais fragmentos.

- Header Checksum – este valor é recalculado para cada fragmento, uma vez que alguns campos do cabeçalho IP mudam durante o processo de fragmentação.

Para reconstruir o datagrama original, o receptor utiliza principalmente três campos do cabeçalho IP: Identification, Fragment Offset e More Fragments (MF). O campo Identification é igual em todos os fragmentos do mesmo datagrama e permite identificar quais fragmentos pertencem ao mesmo pacote original.

Em seguida, o campo Fragment Offset indica a posição correta de cada fragmento dentro do datagrama original, permitindo ordenar os dados corretamente. Finalmente, a flag More Fragments indica se existem mais fragmentos a receber; quando um fragmento chega com MF = 0, o receptor sabe que recebeu o último fragmento. Com base nesta informação, o sistema consegue reorganizar todos os fragmentos e reconstruir o datagrama IP original.

f)

Estime teoricamente o número de fragmentos gerados e o número de bytes transportados em cada um dos fragmentos. Apresente todos os cálculos efetuados, incluindo os campos do cabeçalho IP relevantes para cada um dos fragmentos.

Para estimar teoricamente o número de fragmentos gerados a partir de um datagrama IP original e o número de bytes transportados em cada fragmento, podemos utilizar fórmulas baseadas nas especificações do protocolo IP.

No experimento realizado foi utilizado o comando:

ping -c 4 -s 3662 marco.uminho.pt

O valor -s 3662 corresponde ao número de bytes de dados ICMP enviados no pacote.

No entanto, para obter o tamanho total do datagrama IP é necessário considerar também os cabeçalhos dos protocolos envolvidos.

Assim, temos:

- Dados ICMP = 3662 bytes
- Cabeçalho ICMP = 8 bytes
- Cabeçalho IPv4 = 20 bytes

Logo, o tamanho total do datagrama IP é:

Tamanho do datagrama = 3662+8+20=3690 bytes

Considerando que o MTU (Maximum Transmission Unit) típico das redes Ethernet é:

MTU = 1500 bytes

Cada fragmento IP deve incluir também o cabeçalho IP de 20 bytes. Assim, o número máximo de bytes de dados transportados por fragmento é:

Dados por fragmento = MTU - Tamanho do cabeçalho IP

Dados por fragmento = 1500 - 20 = 1480 bytes

O número de fragmentos pode ser estimado pela seguinte fórmula:

Número de Fragmentos = $\lceil \text{Tamanho total dos dados IP} / (\text{MTU} - \text{Cabeçalho IP}) \rceil$

Neste caso:

$\text{Número de Fragmentos} = \lceil 3670 / (1500 - 20) \rceil = \lceil 3670/1480 \rceil = 3$

Portanto, o datagrama IP original é dividido em três fragmentos.

Dados transportados em cada um dos dois primeiros fragmentos: 1480 bytes

O tamanho do último fragmento pode ser calculado através da seguinte fórmula:

Tamanho do último fragmento = Tamanho total dos dados IP mod (MTU – Cabeçalho IP)
= $3670 \bmod 1480 = 710$ bytes

Teoricamente, o datagrama IP original com 3690 bytes é fragmentado em três fragmentos. Os dois primeiros transportam 1480 bytes de dados, enquanto o último transporta 710 bytes de dados.

Os campos Fragment Offset, More Fragments (MF) e Identification permitem identificar a posição de cada fragmento e garantir que o destinatário consegue reconstruir corretamente o datagrama IP original.

g)

Por que razão apenas o primeiro fragmento de cada pacote é identificado pelo Wireshark como sendo um pacote ICMP?

Apenas o primeiro fragmento de cada pacote é identificado pelo Wireshark como sendo um pacote ICMP porque apenas esse fragmento contém o cabeçalho completo do protocolo ICMP. Quando ocorre fragmentação IP, o datagrama original é dividido em vários fragmentos menores. Cada fragmento contém um cabeçalho IP próprio, mas apenas o primeiro fragmento inclui também o cabeçalho do protocolo de transporte ou de rede superior (neste caso, o cabeçalho ICMP).

Os fragmentos seguintes contêm apenas partes dos dados do pacote original, não incluindo o cabeçalho ICMP. Como resultado, o Wireshark não possui informação suficiente para identificar o protocolo transportado nesses fragmentos e apresenta-os apenas como fragmentos IP.

Assim, apenas o primeiro fragmento pode ser reconhecido pelo Wireshark como um pacote ICMP, enquanto os fragmentos subsequentes são mostrados apenas como fragmentos do protocolo IPv4.

h)

Com que valor é o tamanho do datagrama comparado a fim de se determinar se este deve ser fragmentado?

Quais seriam os efeitos na rede ao aumentar/diminuir este valor?

O tamanho do datagrama IP é comparado com o valor do MTU (Maximum Transmission Unit) da interface de rede. O MTU representa o tamanho máximo de um pacote que pode ser

transmitido através de um enlace sem necessidade de fragmentação. Se o tamanho do datagrama IP for superior ao MTU da rede, o datagrama terá de ser fragmentado em vários fragmentos menores para poder ser transmitido.

Se o valor do MTU for aumentado, torna-se possível transmitir pacotes maiores sem fragmentação. Isto pode melhorar a eficiência da rede, pois reduz o número de fragmentos e o overhead associado aos cabeçalhos adicionais.

Por outro lado, se o MTU for diminuído, aumenta a probabilidade de fragmentação, uma vez que mais pacotes excederão o tamanho máximo permitido. Isto pode gerar mais fragmentos na rede, aumentando o overhead e potencialmente reduzindo a eficiência da transmissão de dados.

Assim, o valor do MTU influencia diretamente o desempenho da rede, uma vez que determina o tamanho máximo dos pacotes que podem ser transmitidos sem fragmentação.

i)

Sabendo que no comando ping a opção “-f” (Windows), “-M do” (Linux) ou “-D” (Mac) ativa a flag “Don’t Fragment” (DF) no cabeçalho do IPv4, usando ping <opção DF> <opção pkt_size> SIZE marco.uminho.pt, (opção pkt_size = -l (Windows) ou -s (Linux, Mac), determine o valor máximo de SIZE sem que ocorra fragmentação do pacote? Justifique o valor obtido.

Para determinar o valor máximo de SIZE sem que ocorra fragmentação, deve ativar-se a flag Don’t Fragment (DF) no cabeçalho IPv4 e testar diferentes tamanhos com o comando ping. Neste caso, o tamanho do pacote enviado pelo ping corresponde apenas aos dados ICMP. No entanto, o datagrama IP inclui também:

- 8 bytes do cabeçalho ICMP
- 20 bytes do cabeçalho IPv4

Considerando que o MTU típico da rede é 1500 bytes, o maior valor possível para SIZE sem fragmentação é dado por:

$\text{SIZE max} = \text{MTU} - \text{Cabeçalho IP} - \text{Cabeçalho ICMP} = 1500 - 20 - 8 = 1472 \text{ bytes} = 1472 \text{ bytes}$

Parte 2

Pergunta 1

Com os avanços da Inteligência Artificial, D. Afonso Henriques termina todas as suas tarefas mais cedo e vê-se com algum tempo livre. Decide então fazer remodelações no reino:

a)

De modo a garantir uma posição estrategicamente mais vantajosa e ter casa de férias para relaxar entre batalhas, ordena a construção de um segundo Castelo, em Braga. Não tendo qualquer queixa do serviço prestado, recorre aos serviços do ISP ReiDaNet, que já utiliza no condado, para ter acesso à rede no segundo Castelo. O ISP atribuiu-lhe o endereço de rede IP 172.26.XX.128/25 em que XX corresponde ao seu número de grupo (PLXX).

Defina um esquema de endereçamento que permita o estabelecimento de pelo menos 6 redes e que garanta que cada uma destas possa ter, pelo menos, 5 hosts. Assuma que todos os endereços de sub-redes são utilizáveis.

Assim que concluído, chame o docente e explique o processo, justificando as suas opções.

RC - TP1 2ª Rede

24

172.26.52.128 / 25

.10090000

Rede Sub-rede

$\log_2(6) = 2,58$

3 bits reservados para as sub-redes

15 hosts por rede = 2 sub-redes = 13 hosts

172.26.52.128 / 25	SA	Range	Range	HL por rede em rede
172.26.52.128 / 28	000	128 - 142	128	0001 - 1110
.144	001	145 - 158	145	
.160	010	161 - 174		
.176	011	177 - 190		
.192	100	193 - 206		
.208	101	209 - 222		
.224	110	225 - 238		
.240	111	241 - 254		

Not atribuídas

O ISP atribuiu ao novo castelo a rede 172.26.52.128/25, sendo necessário definir um esquema de endereçamento que permitisse a criação de, pelo menos, 6 sub-redes, garantindo que cada uma pudesse suportar pelo menos 5 hosts.

Partindo da rede 172.26.52.128/25, existem 7 bits disponíveis para subdivisão e endereçamento de hosts. Para obter pelo menos 6 sub-redes, é necessário reservar 3 bits para sub-redes, pois: $2^3=8$ sub-redes

Ficam então disponíveis 4 bits para hosts em cada sub-rede, o que permite:
 $2^4 = 16$ endereços por sub-rede

Assumindo, como indicado no enunciado, que todos os endereços de sub-rede são utilizáveis, cada sub-rede suporta mais do que os 5 hosts mínimos exigidos.

Assim, a nova máscara passa de /25 para /28.

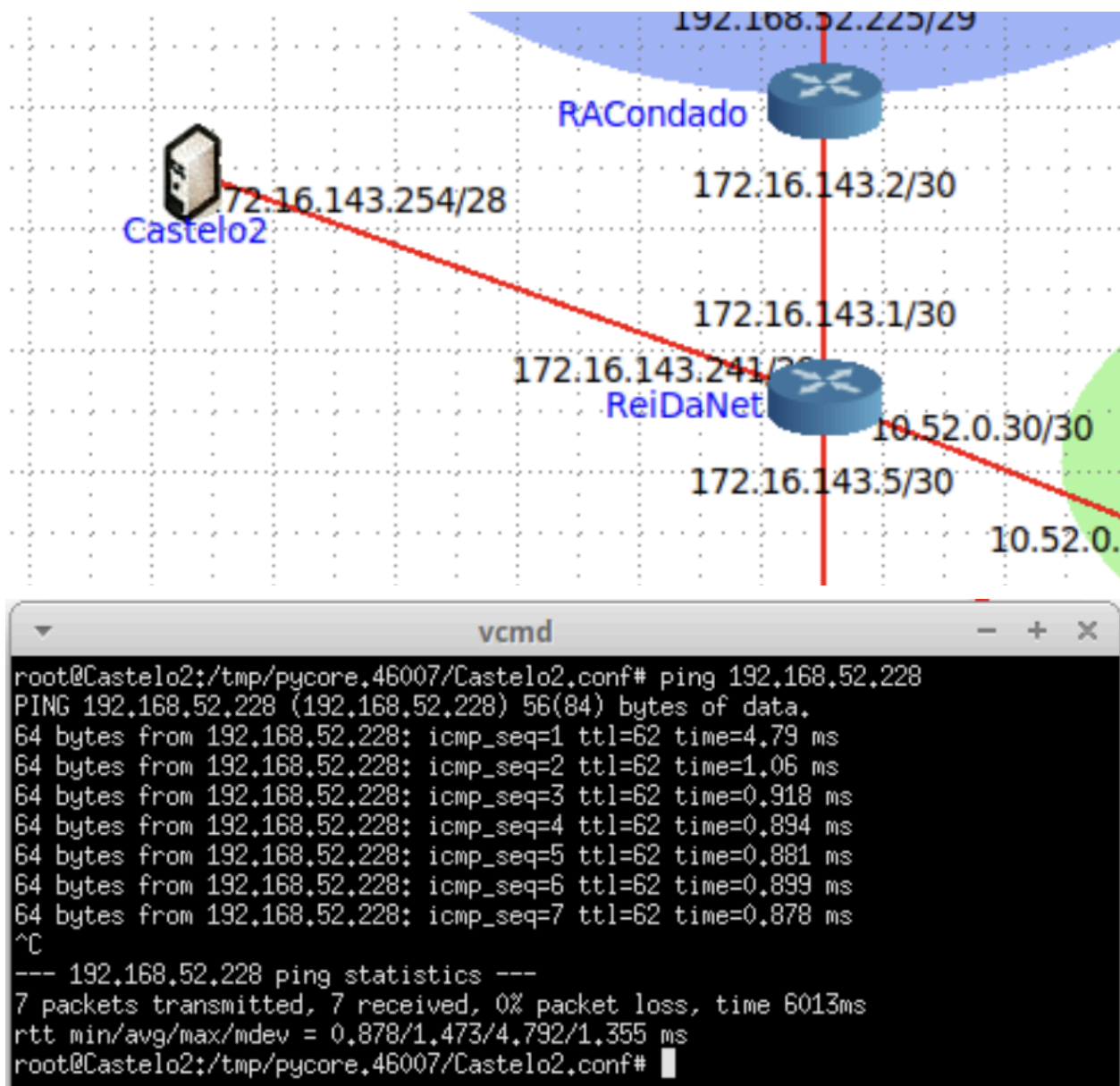
As sub-redes obtidas a partir de 172.26.52.128/25 são as seguintes:

- 172.26.52.128/28
- 172.26.52.144/28
- 172.26.52.160/28
- 172.26.52.176/28
- 172.26.52.192/28
- 172.26.52.208/28
- 172.26.52.224/28
- 172.26.52.240/28

Cada uma destas sub-redes possui 16 endereços, o que satisfaz o requisito do enunciado. Deste modo, conclui-se que a divisão da rede 172.26.52.128/25 em sub-redes /28 constitui uma solução válida para o problema.

b)

Ligue um novo host Castelo2 diretamente ao router ReiDaNet. Associe-lhe um endereço, à sua escolha, pertencente a uma sub-rede disponível das criadas na alínea anterior (garanta que a interface do router ReiDaNet utiliza o primeiro endereço válido da sub-rede escolhida). Verifique que tem conectividade com os dispositivos do Condado Portucalense.



Após a definição do esquema de endereçamento, foi ligado um novo host designado Castelo2 diretamente ao router ReiDaNet.

Foi escolhida uma das sub-redes disponíveis da alínea anterior, concretamente a sub-rede: 172.26.52.240/28

De acordo com o enunciado, a interface do router ReiDaNet deveria utilizar o primeiro endereço válido da sub-rede escolhida.

Assim, foi atribuído ao router o endereço: 172.26.52.241/28

Ao host Castelo2 foi atribuído o endereço: 172.26.52.254/28

Após a configuração dos endereços IP e da ligação entre os dois dispositivos, foi efetuado um teste de conectividade a partir do host Castelo2 para um dispositivo do Condado Portucalense, utilizando o comando ping. O destino utilizado foi o endereço 192.168.52.228.

Os resultados obtidos mostram que todos os pacotes enviados receberam resposta, com 0% de perda, confirmando que existe conectividade entre o novo host e os dispositivos do Condado Portucalense.

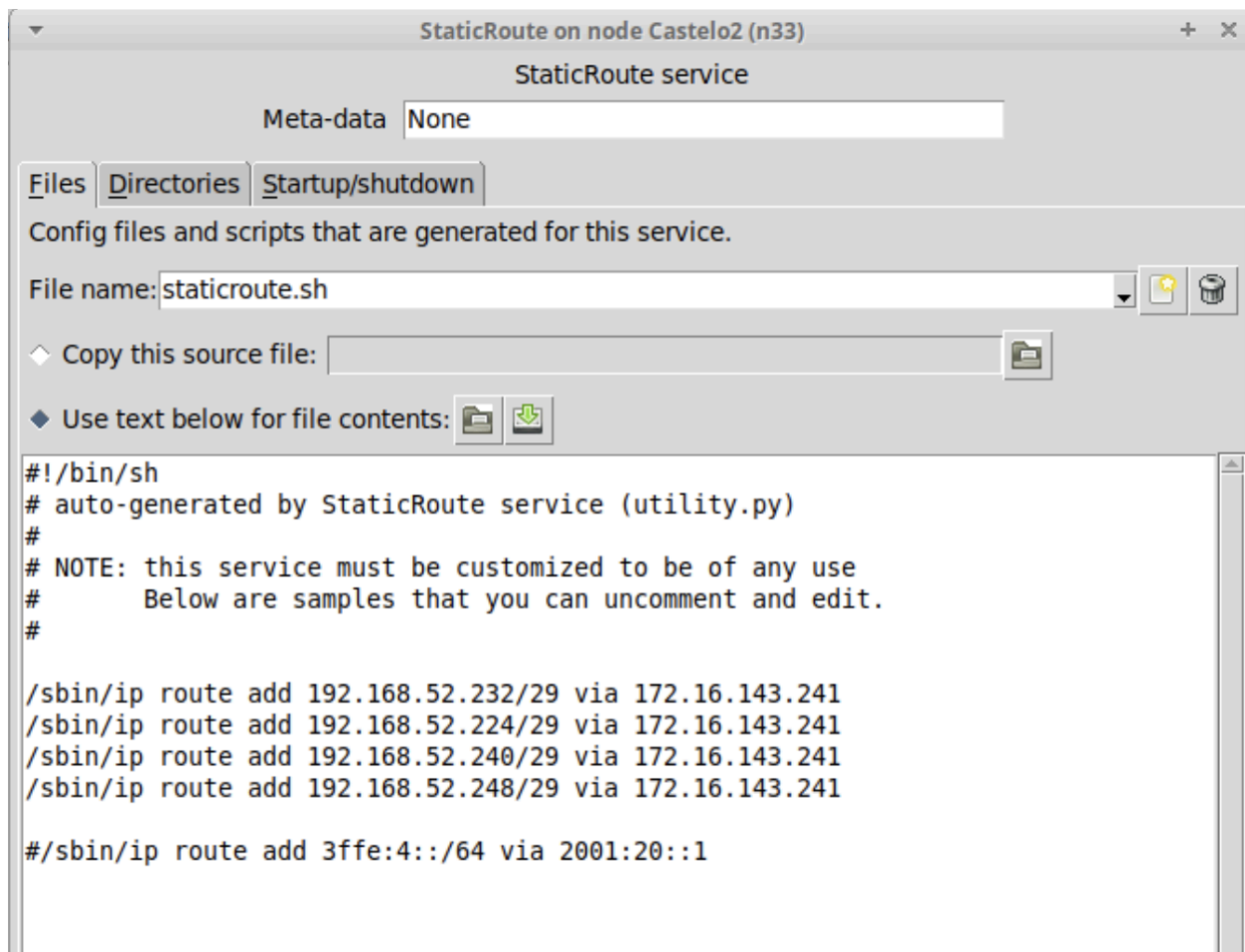
Deste modo, pode concluir-se que a ligação do host Castelo2 ao router ReiDaNet foi corretamente estabelecida, bem como a respetiva configuração de endereçamento IP.

c)

Não estando satisfeito com a decoração deste novo Castelo, opta por eliminar a sua rota default.

Adicione as rotas necessárias para que o Castelo2 continue a ter acesso ao Condado Portucalense e à rede Institucional. Mostre que a conectividade é restabelecida, assim como a tabela de encaminhamento resultante.

Explícite ainda a utilidade de uma rota default.



```

root@Castelo2:/tmp/pycore.46007/Castelo2.conf# ping 192.168.52.252
PING 192.168.52.252 (192.168.52.252) 56(84) bytes of data.
64 bytes from 192.168.52.252: icmp_seq=1 ttl=62 time=0.702 ms
64 bytes from 192.168.52.252: icmp_seq=2 ttl=62 time=0.879 ms
^C
--- 192.168.52.252 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 0.702/0.790/0.879/0.088 ms
root@Castelo2:/tmp/pycore.46007/Castelo2.conf# ping 192.168.52.228
PING 192.168.52.228 (192.168.52.228) 56(84) bytes of data.
64 bytes from 192.168.52.228: icmp_seq=1 ttl=62 time=2.40 ms
64 bytes from 192.168.52.228: icmp_seq=2 ttl=62 time=0.713 ms
^C
--- 192.168.52.228 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 0.713/1.556/2.399/0.843 ms

```

```

--- 192.168.52.234 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 0.953/1.433/2.248/0.579 ms
root@Castelo2:/tmp/pycore.46007/Castelo2.conf# ping 192.168.52.234
PING 192.168.52.234 (192.168.52.234) 56(84) bytes of data.
64 bytes from 192.168.52.234: icmp_seq=1 ttl=62 time=1.14 ms
64 bytes from 192.168.52.234: icmp_seq=2 ttl=62 time=0.439 ms
^C
--- 192.168.52.234 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1003ms
rtt min/avg/max/mdev = 0.439/0.788/1.137/0.349 ms
root@Castelo2:/tmp/pycore.46007/Castelo2.conf# ping 192.168.52.242
PING 192.168.52.242 (192.168.52.242) 56(84) bytes of data.
64 bytes from 192.168.52.242: icmp_seq=1 ttl=62 time=1.07 ms
64 bytes from 192.168.52.242: icmp_seq=2 ttl=62 time=0.985 ms
^C
--- 192.168.52.242 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1003ms
rtt min/avg/max/mdev = 0.985/1.029/1.074/0.044 ms

```

Após a ligação do host Castelo2, foi removida a rota default, com o objetivo de configurar manualmente as rotas necessárias para garantir o acesso às redes do Condado Portucalense e da rede Institucional.

De seguida, foram adicionadas rotas estáticas no host Castelo2, encaminhando o tráfego através do router ReiDaNet, utilizando o seguinte gateway: 172.16.143.241

As rotas configuradas foram:

```
ip route add 192.168.52.232/29 via 172.16.143.241
```

```
ip route add 192.168.52.224/29 via 172.16.143.241
```

```
ip route add 192.168.52.240/29 via 172.16.143.241
ip route add 192.168.52.248/29 via 172.16.143.241
```

Após a configuração destas rotas, foram realizados testes de conectividade a partir do host Castelo2 para diferentes endereços pertencentes às redes do Condado Portucalense, nomeadamente:

- 192.168.52.252
- 192.168.52.228
- 192.168.52.234
- 192.168.52.242

Os resultados obtidos mostram que todos os testes de ping foram bem-sucedidos, com 0% de perda de pacotes, confirmando que a conectividade foi corretamente restabelecida.

A remoção da rota default implica que o host deixa de ter um caminho automático para redes desconhecidas.

Assim, torna-se necessário definir explicitamente rotas para cada rede de destino pretendida. Neste caso, a conectividade foi garantida através da configuração de rotas estáticas específicas.

A rota default tem como principal função encaminhar o tráfego destinado a redes que não estão explicitamente definidas na tabela de encaminhamento. Sem esta rota, o sistema apenas consegue comunicar com redes para as quais existam rotas configuradas manualmente.

Deste modo, verifica-se que, apesar da remoção da rota default, foi possível restabelecer a conectividade desejada através da utilização de rotas estáticas adequadas.

Pergunta 2

D.Afonso Henriques quer enviar fotos do novo Castelo à sua mãe, D.Teresa, mas está a ter alguns problemas de comunicação.

Este alega que o problema deverá estar no dispositivo de D.Teresa, uma vez que no dia anterior conseguiu fazer stream de

Fortnite para todos os seus subscritores da Twitch, e acabou de sair de uma discussão política no Reddit.

a)

Confirme, através do comando ping, que AfonsoHenriques tem efetivamente conectividade com os servidores Reddit e Twitch.

```

rtt min/avg/max/mdev = 0.533/3.494/5.456/1.961 ms
<pycore.44295/AfonsoHenriques.conf# ping 192.168.0.155
PING 192.168.0.155 (192.168.0.155) 56(84) bytes of data.
64 bytes from 192.168.0.155: icmp_seq=1 ttl=55 time=5.46 ms
64 bytes from 192.168.0.155: icmp_seq=2 ttl=55 time=1.53 ms
^C
--- 192.168.0.155 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 1.533/3.494/5.456/1.961 ms
root@AfonsoHenriques:/tmp/pycore.44295/AfonsoHenriques.conf# ping 192.168.146
PING 192.168.146 (192.168.0.146) 56(84) bytes of data.
64 bytes from 192.168.0.146: icmp_seq=1 ttl=55 time=2.11 ms
64 bytes from 192.168.0.146: icmp_seq=2 ttl=55 time=1.83 ms
^C
--- 192.168.146 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms

```

Para confirmar a conectividade do host AfonsoHenriques com os servidores Reddit e Twitch, foram realizados testes utilizando o comando ping para os respectivos endereços IP.

Os resultados obtidos mostram que, em ambos os casos, os pacotes enviados foram recebidos com sucesso, não se verificando qualquer perda de pacotes (0% packet loss). Foram obtidas respostas do tipo:

64 bytes from 192.168.0.155

64 bytes from 192.168.0.146

Estes resultados indicam que o host AfonsoHenriques consegue comunicar corretamente com ambos os servidores, confirmando que existe conectividade entre os dispositivos.

b)

Recorrendo ao comando netstat -rn, analise as tabelas de encaminhamento dos dispositivos AfonsoHenriques e Teresa. Existe algum problema com as suas entradas? Identifique e descreva a utilidade de cada uma das entradas destes dois hosts.

```

root@AfonsoHenriques:/tmp/pycore.46007/AfonsoHenriques.conf# netstat -rn
Kernel IP routing table
Destination      Gateway         Genmask         Flags   MSS Window  irtt Iface
0.0.0.0          192.168.52.225 0.0.0.0         UG        0 0          0 eth0
192.168.52.224  0.0.0.0         255.255.255.248 U        0 0          0 eth0
root@AfonsoHenriques:/tmp/pycore.46007/AfonsoHenriques.conf# █

```

```

root@Teresa:/tmp/pycore.40217/Teresa.conf# netstat -rn
Kernel IP routing table
Destination      Gateway         Genmask         Flags   MSS Window  irtt Iface
0.0.0.0          192.168.52.129 0.0.0.0         UG        0 0          0 eth0
192.168.52.128  0.0.0.0         255.255.255.248 U        0 0          0 eth0
root@Teresa:/tmp/pycore.40217/Teresa.conf# █

```

Através do comando `netstat -rn`, foram analisadas as tabelas de encaminhamento dos hosts AfonsoHenriques e Teresa.

1) AfonsoHenriques

A tabela de encaminhamento apresenta as seguintes entradas:

- 0.0.0.0 via 192.168.52.225
Esta é a rota default, que encaminha todo o tráfego destinado a redes não explicitamente presentes na tabela. O gateway associado (192.168.52.225) corresponde ao router da rede local.
- 192.168.52.224/29 via 0.0.0.0
Esta entrada representa a rede local diretamente ligada ao host. Como o gateway é 0.0.0.0, significa que os destinos pertencentes a esta rede são alcançados diretamente através da interface `eth0`.

2) Teresa

A tabela de encaminhamento apresenta igualmente duas entradas:

- 0.0.0.0 via 192.168.52.129
Esta é a rota default do host Teresa, utilizada para encaminhar tráfego destinado a redes externas. O gateway (192.168.52.129) corresponde ao router da sua rede local.
- 192.168.52.128/29 via 0.0.0.0
Esta entrada corresponde à rede local diretamente ligada ao host Teresa, sendo acessível diretamente através da interface `eth0`.

As tabelas de encaminhamento de ambos os hosts estão corretamente configuradas do ponto de vista local, uma vez que:

- cada host possui uma rota para a sua rede local;
- cada host possui uma rota default para comunicação com redes externas.

No entanto, apesar destas entradas estarem corretas, verificou-se na alínea (a) que não existe conectividade com os servidores Reddit e Twitch, surgindo a mensagem “Destination Net Unreachable”.

Isto indica que, embora os hosts estejam corretamente configurados, o problema encontra-se no encaminhamento ao nível dos routers da rede, que não possuem rotas adequadas para alcançar as redes de destino.

Assim, conclui-se que não existe erro nas tabelas de encaminhamento dos hosts, mas sim uma falha no encaminhamento em dispositivos intermédios da rede.

c)

Analise o comportamento dos routers do core da rede (n1 a n6) quando tenta estabelecer comunicação entre os hosts AfonsoHenriques e Teresa. Indique que dispositivo(s) não permite(m) o encaminhamento correto dos pacotes. Seguidamente, avalie e explique a(s) causa(s) do funcionamento incorreto do dispositivo.

Utilize o comando `ip route add/del` para adicionar as rotas necessárias ou remover rotas incorretas. Verifique a sintaxe completa do comando a usar com `man ip-route` ou `man`

route. Poderá também utilizar o comando traceroute para se certificar do caminho nó a nó. Considere a alínea resolvida assim que houver tráfego a chegar ao ISP CondadOnline.

```
<pycore.44295/AfonsoHenriques.conf# traceroute 192.168.0.130
traceroute to 192.168.0.130 (192.168.0.130), 30 hops max, 60 byte packets
 1 192.168.0.225 (192.168.0.225) 3.176 ms 0.828 ms 0.453 ms
 2 172.16.143.1 (172.16.143.1) 0.649 ms 0.473 ms 0.606 ms
 3 10.0.0.29 (10.0.0.29) 0.528 ms 0.376 ms 0.374 ms
 4 10.0.0.25 (10.0.0.25) 0.702 ms 0.485 ms 0.446 ms
 5 10.0.0.25 (10.0.0.25) 3074.665 ms !H 3074.094 ms !H 3073.808 ms !H
root@AfonsoHenriques:/tmp/pycore.44295/AfonsoHenriques.conf#
```

Destination	Gateway	Genmask	Flags	Metric	Ref	Use	Iface
10.0.0.0	10.0.0.13	255.255.255.252	UG	0	0	0	eth1
10.0.0.4	10.0.0.21	255.255.255.252	UG	0	0	0	eth0
10.0.0.8	10.0.0.13	255.255.255.252	UG	0	0	0	eth1
10.0.0.12	0.0.0.0	255.255.255.252	U	0	0	0	eth1
10.0.0.16	10.0.0.13	255.255.255.252	UG	0	0	0	eth1
10.0.0.20	0.0.0.0	255.255.255.252	U	0	0	0	eth0
10.0.0.24	0.0.0.0	255.255.255.252	U	0	0	0	eth2
10.0.0.28	10.0.0.26	255.255.255.252	UG	0	0	0	eth2
172.0.0.0	10.0.0.26	255.0.0.0	UG	0	0	0	eth2
172.16.142.0	10.0.0.13	255.255.255.252	UG	0	0	0	eth1
172.16.142.4	10.0.0.21	255.255.255.252	UG	0	0	0	eth0
172.16.143.0	10.0.0.26	255.255.255.252	UG	0	0	0	eth2
172.16.143.4	10.0.0.26	255.255.255.252	UG	0	0	0	eth2
192.168.0.128	10.0.0.13	255.255.255.248	UG	0	0	0	eth1
192.168.0.130	10.0.0.25	255.255.255.254	UG	0	0	0	eth2
192.168.0.136	10.0.0.21	255.255.255.248	UG	0	0	0	eth0
192.168.0.144	10.0.0.21	255.255.255.248	UG	0	0	0	eth0
192.168.0.152	10.0.0.21	255.255.255.248	UG	0	0	0	eth0
192.168.0.224	10.0.0.26	255.255.255.248	UG	0	0	0	eth2
192.168.0.232	10.0.0.26	255.255.255.248	UG	0	0	0	eth2
192.168.0.240	10.0.0.26	255.255.255.248	UG	0	0	0	eth2
192.168.0.248	10.0.0.26	255.255.255.248	UG	0	0	0	eth2

```
root@n2:/tmp/pycore.44295/n2.conf#
```

Podemos ver que quando é ping para 192.168.0.130 ele está a usar o gateway dele próprio, ficando preso em loop nele.

Remover a entrada do 192.168.0.130 corrige este problema pois assim o ip da teresa vai para a regra mais abrangente da submask 248 no 128.


```

<pycore.44295/AfonsoHenriques.conf# traceroute 192.168.0.130
traceroute to 192.168.0.130 (192.168.0.130), 30 hops max, 60 byte packets
 1 192.168.0.225 (192.168.0.225) 1.325 ms 0.269 ms 0.245 ms
 2 172.16.143.1 (172.16.143.1) 1.134 ms 0.354 ms 0.354 ms
 3 10.0.0.29 (10.0.0.29) 0.826 ms 0.654 ms 0.430 ms
 4 10.0.0.25 (10.0.0.25) 0.908 ms 0.433 ms 0.766 ms
 5 10.0.0.13 (10.0.0.13) 1.067 ms 0.555 ms 0.501 ms
 6 10.0.0.25 (10.0.0.25) 0.822 ms 0.508 ms 0.411 ms
 7 10.0.0.13 (10.0.0.13) 0.234 ms 0.222 ms 0.261 ms
 8 * * *
 9 * * *
10 * * *
11 * * *
12 * * *
13 * 10.0.0.13 (10.0.0.13) 1.575 ms 1.124 ms
14 10.0.0.25 (10.0.0.25) 1.072 ms 1.092 ms 0.752 ms
15 10.0.0.13 (10.0.0.13) 1.434 ms 0.949 ms 1.816 ms
16 10.0.0.25 (10.0.0.25) 0.768 ms 0.701 ms *^C
root@AfonsoHenriques:/tmp/pycore.44295/AfonsoHenriques.conf# █

```

```

Kernel IP routing table
Destination      Gateway         Genmask         Flags Metric Ref    Use Iface
10.0.0.0         10.0.0.9       255.255.255.252 UG          0      0        0 eth0
10.0.0.4         10.0.0.9       255.255.255.252 UG          0      0        0 eth0
10.0.0.8         0.0.0.0        255.255.255.252 U          0      0        0 eth0
10.0.0.12        0.0.0.0        255.255.255.252 U          0      0        0 eth1
10.0.0.16        10.0.0.9       255.255.255.252 UG          0      0        0 eth0
10.0.0.20        10.0.0.14      255.255.255.252 UG          0      0        0 eth1
10.0.0.24        10.0.0.14      255.255.255.252 UG          0      0        0 eth1
10.0.0.28        10.0.0.14      255.255.255.252 UG          0      0        0 eth1
172.0.0.0        10.0.0.14      255.0.0.0       UG          0      0        0 eth1
172.16.142.0     10.0.0.9       255.255.255.252 UG          0      0        0 eth0
172.16.142.4     10.0.0.9       255.255.255.252 UG          0      0        0 eth0
172.16.143.0     10.0.0.14      255.255.255.252 UG          0      0        0 eth1
172.16.143.4     10.0.0.14      255.255.255.252 UG          0      0        0 eth1
192.168.0.128    10.0.0.14      255.255.255.248 UG          0      0        0 eth1
192.168.0.136    10.0.0.9       255.255.255.248 UG          0      0        0 eth0
192.168.0.144    10.0.0.9       255.255.255.248 UG          0      0        0 eth0
192.168.0.152    10.0.0.9       255.255.255.248 UG          0      0        0 eth0
192.168.0.224    10.0.0.14      255.255.255.248 UG          0      0        0 eth1
192.168.0.232    10.0.0.14      255.255.255.248 UG          0      0        0 eth1
192.168.0.240    10.0.0.14      255.255.255.248 UG          0      0        0 eth1
192.168.0.248    10.0.0.14      255.255.255.248 UG          0      0        0 eth1
root@n1:/tmp/pycore.44295/n1.conf# █

```

Agora conseguimos verificar que o packet fica em preso num loop no n1.


```
ip route 172.0.0.0 255.0.0.0 10.0.0.14
ip route 192.168.0.128 255.255.255.248 10.0.0.9
ip route 192.168.0.152 255.255.255.248 10.0.0.0
```

Com a alteração desta entrada para evitar o loop

```
ip route 192.168.0.128 255.255.255.248 10.0.0.5
ip route 192.168.0.152 255.255.255.248 10.0.0.5
```

e a adição desta entrada no n3 vamos refazer o traceroute.

```
<pycore.44295/AfonsoHenriques.conf# traceroute 192.168.0.130
traceroute to 192.168.0.130 (192.168.0.130), 30 hops max, 60 byte packets
 1 192.168.0.225 (192.168.0.225) 2.011 ms 0.249 ms 0.234 ms
 2 172.16.143.1 (172.16.143.1) 0.810 ms 0.693 ms 0.323 ms
 3 10.0.0.29 (10.0.0.29) 0.876 ms 0.392 ms 0.352 ms
 4 10.0.0.25 (10.0.0.25) 1.285 ms 0.488 ms 0.439 ms
 5 10.0.0.13 (10.0.0.13) 1.386 ms 0.521 ms 0.483 ms
 6 10.0.0.17 (10.0.0.17) 1.780 ms 0.614 ms 0.291 ms
 7 10.0.0.5 (10.0.0.5) 1.064 ms 0.355 ms 0.265 ms
 8 10.0.0.1 (10.0.0.1) 0.543 ms 0.312 ms 0.284 ms
 9 * * *
10 * * *
11 * * *
12 * * *
13 * * *
14 *^C
root@AfonsoHenriques:/tmp/pycore.44295/AfonsoHenriques.conf#
```

Com este último traceroute verificamos que já conseguimos alcançar o ISP CondadOnline.

d)

Uma vez que o core da rede esteja a encaminhar corretamente os pacotes enviados por AfonsoHenriques, confira com o Wireshark se estes são recebidos por Teresa.

Através do Wireshark, foi possível confirmar que os pacotes enviados pelo host AfonsoHenriques são efetivamente recebidos por Teresa. Na captura observam-se pacotes do tipo ICMP Echo Request com origem em 192.168.0.226 e destino em 192.168.0.130, o que demonstra que o encaminhamento no sentido AfonsoHenriques → Teresa está a funcionar corretamente.

No entanto, apesar de os pacotes chegarem a Teresa, verificou-se inicialmente que o comando ping apresentava 100% de perda de pacotes, indicando ausência de conectividade. Isto

significa que o problema não se encontrava no envio dos pacotes, mas sim no caminho de retorno (resposta).

```
49 42.836419863 192.168.0.226 192.168.0.130 ICMP 98 Echo (ping) request id=0x0026, seq=6/1536, ttl=55 (repl
50 42.836572675 192.168.0.130 192.168.0.226 ICMP 98 Echo (ping) reply id=0x0026, seq=6/1536, ttl=64 (requ
51 43.857646219 192.168.0.226 192.168.0.130 ICMP 98 Echo (ping) request id=0x0026, seq=7/1792, ttl=55 (repl

<ycore.44295/AfonsoHenriques.conf# ping 192.168.0.130
PING 192.168.0.130 (192.168.0.130) 56(84) bytes of data.
^C
--- 192.168.0.130 ping statistics ---
7 packets transmitted, 0 received, 100% packet loss, time 6128ms

root@AfonsoHenriques:/tmp/pycore.44295/AfonsoHenriques.conf#
```

```
41 40.785896094 192.168.0.129 192.168.0.130 ICMP 126 Destination unreachable (Network unreachable)
42 41.800514000 192.168.0.129 192.168.0.130 ICMP 98 Echo (ping) request id=0x0026, seq=5/1400, ttl=55
```

i)

Em caso afirmativo, porque é que continua a não existir conectividade entre D.Teresa e D.Afonso Henriques?

Efetue as alterações necessárias para garantir que a conectividade é restabelecida e o confronto entre os dois é evitado.

```
traceroute to 192.168.0.226 (192.168.0.226), 30 hops max, 60 byte packets
 1 192.168.0.129 (192.168.0.129) 0.673 ms !N 0.234 ms !N *
```

Podemos observar através do traceroute da Teresa até o Afonso que não passa pelo RAGaliza.

A ausência de conectividade deve-se ao facto de os pacotes de resposta (ICMP Echo Reply) não conseguirem regressar ao host de origem.

Isto foi confirmado com o comando traceroute a partir de Teresa, onde inicialmente se observou:

1 192.168.0.129 ... !N

A indicação !N (Network Unreachable) demonstra que o gateway de Teresa não possuía uma rota válida para alcançar a rede de AfonsoHenriques. Assim, embora os pacotes ICMP Echo Request chegassem a Teresa, as respostas não conseguiam ser encaminhadas de volta.

Após a análise da tabela de encaminhamento do router RAGaliza, foram efetuadas as alterações necessárias às rotas estáticas, permitindo estabelecer um caminho válido no sentido de retorno.

Depois da correção, o traceroute passou a completar-se com sucesso, restabelecendo a conectividade entre os dois hosts.

```

root@RAGaliza:/tmp/pycore.44295/RAGaliza.conf# route
Kernel IP routing table
Destination      Gateway          Genmask          Flags Metric Ref    Use Iface
10.0.0.0          172.16.142.1    255.255.255.252 UG        0      0        0 eth0
10.0.0.4          172.16.142.1    255.255.255.252 UG        0      0        0 eth0
10.0.0.8          172.16.142.1    255.255.255.252 UG        0      0        0 eth0
10.0.0.12         172.16.142.1    255.255.255.252 UG        0      0        0 eth0
10.0.0.16         172.16.142.1    255.255.255.252 UG        0      0        0 eth0
10.0.0.20         172.16.142.1    255.255.255.252 UG        0      0        0 eth0
10.0.0.24         172.16.142.1    255.255.255.252 UG        0      0        0 eth0
10.0.0.28         172.16.142.1    255.255.255.252 UG        0      0        0 eth0
172.0.0.0         172.16.142.1    255.0.0.0        UG        0      0        0 eth0
172.16.142.0      0.0.0.0          255.255.255.252 U         0      0        0 eth0
172.16.142.4      172.16.142.1    255.255.255.252 UG        0      0        0 eth0
172.16.143.0      172.16.142.1    255.255.255.252 UG        0      0        0 eth0
172.16.143.4      172.16.142.1    255.255.255.252 UG        0      0        0 eth0
192.168.0.128     0.0.0.0          255.255.255.248 U         0      0        0 eth1
192.168.0.136     172.16.142.1    255.255.255.248 UG        0      0        0 eth0
192.168.0.144     172.16.142.1    255.255.255.248 UG        0      0        0 eth0
192.168.0.152     172.16.142.1    255.255.255.248 UG        0      0        0 eth0
192.168.0.192     0.0.0.0          255.255.255.248 U         0      0        0 eth1
192.168.0.200     172.16.142.1    255.255.255.248 UG        3      0        0 eth0
192.168.0.208     172.16.142.1    255.255.255.248 UG        3      0        0 eth0
192.168.0.216     172.16.142.1    255.255.255.248 UG        3      0        0 eth0
192.168.0.232     172.16.142.1    255.255.255.248 UG        0      0        0 eth0
192.168.0.240     172.16.142.1    255.255.255.248 UG        0      0        0 eth0
192.168.0.248     172.16.142.1    255.255.255.248 UG        0      0        0 eth0
root@RAGaliza:/tmp/pycore.44295/RAGaliza.conf#

```

```

root@Teresa:/tmp/pycore.44295/Teresa.conf# traceroute 192.168.0.226
traceroute to 192.168.0.226 (192.168.0.226), 30 hops max, 60 byte packets
 1 192.168.0.129 (192.168.0.129) 1.633 ms 0.281 ms 0.335 ms
 2 172.16.142.1 (172.16.142.1) 0.878 ms 0.625 ms 0.335 ms
 3 10.0.0.2 (10.0.0.2) 0.924 ms 0.399 ms 0.505 ms
 4 10.0.0.6 (10.0.0.6) 1.128 ms 0.513 ms 0.500 ms
 5 10.0.0.18 (10.0.0.18) 1.520 ms 0.582 ms 0.557 ms
 6 10.0.0.14 (10.0.0.14) 2.040 ms 0.536 ms 0.295 ms
 7 10.0.0.26 (10.0.0.26) 1.401 ms 0.380 ms 0.307 ms
 8 10.0.0.30 (10.0.0.30) 0.626 ms 0.463 ms 0.421 ms
 9 172.16.143.2 (172.16.143.2) 0.822 ms 0.394 ms 0.396 ms
10 192.168.0.226 (192.168.0.226) 0.714 ms 0.468 ms 0.429 ms
root@Teresa:/tmp/pycore.44295/Teresa.conf#

```

```

pycore.44295/AfonsoHenriques.conf# traceroute 192.168.0.130
traceroute to 192.168.0.130 (192.168.0.130), 30 hops max, 60 byte packets
 1 192.168.0.225 (192.168.0.225) 1.182 ms 0.284 ms 0.206 ms
 2 172.16.143.1 (172.16.143.1) 0.702 ms 0.280 ms 0.287 ms
 3 10.0.0.29 (10.0.0.29) 0.444 ms 0.349 ms 0.366 ms
 4 10.0.0.25 (10.0.0.25) 0.682 ms 0.394 ms 0.403 ms
 5 10.0.0.13 (10.0.0.13) 0.582 ms 0.485 ms 0.687 ms
 6 10.0.0.17 (10.0.0.17) 0.646 ms 0.518 ms 0.220 ms
 7 10.0.0.5 (10.0.0.5) 0.305 ms 0.230 ms 0.235 ms
 8 10.0.0.1 (10.0.0.1) 0.310 ms 0.294 ms 0.389 ms
 9 172.16.142.2 (172.16.142.2) 0.366 ms 0.290 ms 0.317 ms
10 192.168.0.130 (192.168.0.130) 0.398 ms 0.350 ms 0.329 ms
root@AfonsoHenriques:/tmp/pycore.44295/AfonsoHenriques.conf#

```

ii)

As rotas dos pacotes ICMP echo reply são as mesmas, mas em sentido inverso, que as rotas dos pacotes ICMP echo request enviados entre AfonsoHenriques e Teresa? (Sugestão: analise as rotas nos dois sentidos com o traceroute). Mostre graficamente a rota seguida nos dois sentidos por esses pacotes ICMP.

Através da utilização do comando traceroute nos dois sentidos, foi possível analisar os percursos seguidos pelos pacotes.

Comparando os dois percursos, verifica-se que não são iguais em sentido inverso, ou seja, os pacotes seguem caminhos diferentes em cada direção.

Isto demonstra a existência de encaminhamento assimétrico, situação comum em redes com múltiplos caminhos e rotas estáticas distintas.

e)

Estando restabelecida a conectividade entre os dois hosts, obtenha a tabela de encaminhamento de n5 e foque-se na seguinte entrada:

ip route 192.168.GrupoID.0 255.255.255.0. 10.0.0.30

Existe uma correspondência (match) nesta entrada para pacotes enviados para o polo Galiza? E para CDN? Caso seja essa a entrada utilizada para o encaminhamento, permitirá o funcionamento esperado do dispositivo?

Ofereça uma explicação pela qual essa entrada é ou não utilizada.

```

root@n5:/tmp/pycore.44295/n5.conf# route
Kernel IP routing table
Destination      Gateway         Genmask         Flags Metric Ref    Use Iface
10.0.0.0         10.0.0.25      255.255.255.252 UG      0     0      0 eth1
10.0.0.4         10.0.0.25      255.255.255.252 UG      0     0      0 eth1
10.0.0.8         10.0.0.25      255.255.255.252 UG      0     0      0 eth1
10.0.0.12        10.0.0.25      255.255.255.252 UG      0     0      0 eth1
10.0.0.16        10.0.0.25      255.255.255.252 UG      0     0      0 eth1
10.0.0.20        10.0.0.25      255.255.255.252 UG      0     0      0 eth1
10.0.0.24        0.0.0.0        255.255.255.252 U       0     0      0 eth1
10.0.0.28        0.0.0.0        255.255.255.252 U       0     0      0 eth0
172.0.0.0        10.0.0.30      255.0.0.0      UG      0     0      0 eth0
172.16.142.0     10.0.0.25      255.255.255.248 UG      0     0      0 eth1
172.16.143.0     10.0.0.30      255.255.255.252 UG      2     0      0 eth0
172.16.143.0     10.0.0.30      255.255.255.248 UG      0     0      0 eth0
172.16.143.4     10.0.0.30      255.255.255.252 UG      2     0      0 eth0
192.142.0.4      10.0.0.25      255.255.255.252 UG      0     0      0 eth1
192.168.0.0      10.0.0.30      255.255.255.0  UG      0     0      0 eth0
192.168.0.128    10.0.0.25      255.255.255.248 UG      0     0      0 eth1
192.168.0.136    10.0.0.25      255.255.255.248 UG      0     0      0 eth1
192.168.0.144    10.0.0.25      255.255.255.248 UG      0     0      0 eth1
192.168.0.152    10.0.0.25      255.255.255.248 UG      0     0      0 eth1
192.168.0.224    10.0.0.30      255.255.255.248 UG      3     0      0 eth0
192.168.0.232    10.0.0.30      255.255.255.248 UG      3     0      0 eth0
192.168.0.240    10.0.0.30      255.255.255.248 UG      3     0      0 eth0
192.168.0.248    10.0.0.30      255.255.255.248 UG      3     0      0 eth0
root@n5:/tmp/pycore.44295/n5.conf# █

```

Após o restabelecimento da conectividade entre os hosts, foi analisada a tabela de encaminhamento do router **n5**, focando na seguinte entrada:

192.168.0.0 255.255.255.0 via 10.0.0.30

Correspondência (match)

Esta entrada corresponde à rede 192.168.0.0/24, pelo que abrange todos os endereços no intervalo 192.168.0.x.

- Polo Galiza:
As redes do polo Galiza pertencem a sub-redes dentro deste intervalo (por exemplo, /29). Assim, existe correspondência com esta entrada.
- CDN:
As redes da CDN também pertencem ao intervalo 192.168.0.x, pelo que também existe correspondência.

Utilização da entrada no encaminhamento

Apesar de existir correspondência, esta entrada não é utilizada para o encaminhamento na maioria dos casos.

Isto deve-se ao princípio do Longest Prefix Match, segundo o qual o router escolhe sempre a rota mais específica (com máscara maior).

Na tabela do n5 existem entradas mais específicas, como por exemplo:

192.168.0.128/29

192.168.0.136/29

192.168.0.144/29

192.168.0.224/29

...

Estas rotas (/29) são mais específicas do que a rota /24, sendo por isso preferidas no processo de encaminhamento.

Impacto no funcionamento

Se esta entrada (/24) fosse utilizada, poderia levar a encaminhamento incorreto dos pacotes, uma vez que o gateway indicado poderia não ser o mais adequado para todas as sub-redes. No entanto, como existem rotas mais específicas na tabela, esta entrada funciona apenas como rota genérica de fallback, não comprometendo o funcionamento normal do router.

Conclui-se que:

- existe correspondência (match) desta entrada tanto para o polo Galiza como para a CDN;
- contudo, esta entrada não é utilizada, pois existem rotas mais específicas na tabela;
- o router aplica o critério de Longest Prefix Match, garantindo o encaminhamento correto;
- assim, o funcionamento do dispositivo não é afetado por esta entrada.

f)

Os endereços utilizados pelos quatro polos são endereços públicos ou privados? E os utilizados no core da rede/ISPs? Justifique convenientemente.

Os endereços utilizados nos quatro polos são endereços privados.

Isto porque pertencem a intervalos reservados definidos pelo RFC 1918, nomeadamente:

- 10.0.0.0/8
- 172.16.0.0 – 172.31.255.255 (/12)
- 192.168.0.0/16

Pelos teus prints, vemos endereços como:

- 10.x.x.x
- 172.16.x.x
- 192.168.x.x

Logo, são claramente endereços privados.

Os endereços utilizados no core da rede/ISPs também são privados:

- Também pertencem aos mesmos intervalos privados (ex: 10.x.x.x e 172.16.x.x)
- Isto é comum em redes internas/simuladas/laboratoriais
- Evita desperdício de endereços públicos

Todos os endereços (polos e core) são privados, pois pertencem a intervalos reservados para redes internas e não são roteáveis diretamente na Internet.

g)

Os switches localizados em cada um dos polos têm um endereço IP atribuído? Porquê?

Os switches não têm necessariamente endereço IP atribuído:

- Um switch funciona na camada 2 (Data Link) do modelo OSI
- Ele encaminha frames com base em endereços MAC, não em IP
- Por isso, não precisa de IP para funcionar

Um switch pode ter IP apenas para:

- gestão/administração (ex: SSH, web, ping)
- Monitorização

Mas isso é opcional e não influencia o encaminhamento de tráfego.

Pergunta 3

Ao ver as fotos no CondadoGram, D. Teresa não ficou convencida com as novas alterações e ordena que Afonso Henriques vá arrumar o castelo. Inconformado, este decide planejar um novo ataque, mas constata que o seu exército não só perde bastante tempo a decidir que direção tomar a cada salto como, por vezes, inclusivamente se perde.

De modo a facilitar a travessia, elimine as rotas referentes a Galiza e CDN no dispositivo n6 e defina um esquema de sumarização de rotas (Supernetting) que permita o uso de apenas uma rota para ambos os polos. Confirme que a conectividade é mantida e chame o docente para explicar o processo efetuado.

```
ip route 192.168.0.128 255.255.255.248 10.0.0.1
ip route 192.168.0.152 255.255.255.248 10.0.0.1
ip route 192.168.0.144 255.255.255.248 10.0.0.1
ip route 192.168.0.136 255.255.255.248 10.0.0.1
ip route 172.16.142.0 255.255.255.252 10.0.0.1
ip route 172.16.142.4 255.255.255.252 10.0.0.1
```

Estas são as rotas pré definidas no n6 referentes à Galiza e ao CDN. Podemos Observar que as redes [192.168.0.xxx](#) são facilmente englobadas numa só seguindo a máscara 255.255.255.224, uma transformação de /29 em um /27. Isto é aplicável pois mesmo que exista alguma rede com uma máscara que seja similar a tabela de forwarding vai seguir a de maior

especificidade. As regras foram substituídas pela seguinte:

```
192.168.0.128 10.0.0.1 255.255.255.224 UG 0 0 0 eth0
```

```
<ycore.46339/AfonsoHenriques.conf# traceroute 192.168.130
traceroute to 192.168.130 (192.168.0.130), 30 hops max, 60 byte packets
 1 192.168.0.225 (192.168.0.225) 10.577 ms 0.362 ms 0.103 ms
 2 172.16.143.1 (172.16.143.1) 0.771 ms 0.126 ms 0.117 ms
 3 10.0.0.29 (10.0.0.29) 0.359 ms 0.144 ms 0.137 ms
 4 10.0.0.25 (10.0.0.25) 0.626 ms 0.173 ms 0.169 ms
 5 10.0.0.13 (10.0.0.13) 0.440 ms 0.202 ms 0.197 ms
 6 10.0.0.17 (10.0.0.17) 0.723 ms 0.549 ms 0.462 ms
 7 10.0.0.5 (10.0.0.5) 0.751 ms 0.284 ms 0.267 ms
 8 10.0.0.1 (10.0.0.1) 0.526 ms 0.440 ms 0.581 ms
 9 172.16.142.2 (172.16.142.2) 0.628 ms 0.364 ms 0.354 ms
10 192.168.0.130 (192.168.0.130) 0.914 ms 0.403 ms 0.395 ms
root@AfonsoHenriques:/tmp/pycore.46339/AfonsoHenriques.conf#
```

```
<ycore.46339/AfonsoHenriques.conf# traceroute 192.168.0.154
traceroute to 192.168.0.154 (192.168.0.154), 30 hops max, 60 byte packets
 1 192.168.0.225 (192.168.0.225) 0.591 ms 0.236 ms 0.274 ms
 2 172.16.143.1 (172.16.143.1) 0.381 ms 0.294 ms 0.300 ms
 3 10.0.0.29 (10.0.0.29) 0.452 ms 0.425 ms 0.365 ms
 4 10.0.0.25 (10.0.0.25) 0.509 ms 0.427 ms 0.425 ms
 5 10.0.0.21 (10.0.0.21) 0.621 ms 0.498 ms 0.478 ms
 6 10.0.0.17 (10.0.0.17) 0.678 ms 0.402 ms 0.218 ms
 7 10.0.0.5 (10.0.0.5) 0.480 ms 0.247 ms 0.245 ms
 8 10.0.0.1 (10.0.0.1) 0.306 ms 0.275 ms 0.264 ms
 9 172.16.142.6 (172.16.142.6) 0.373 ms 0.350 ms 0.303 ms
10 192.168.0.154 (192.168.0.154) 0.663 ms 0.370 ms 0.376 ms
root@AfonsoHenriques:/tmp/pycore.46339/AfonsoHenriques.conf#
```

Confirmamos assim que a conectividade é mantida.

A utilização de supernetting permite reduzir o número de entradas na tabela de encaminhamento, contribuindo para uma maior eficiência no processamento de pacotes pelos routers e melhorando a escalabilidade da rede.

Conclusão

Ao longo deste trabalho prático foi possível aprofundar diversos conceitos fundamentais das redes de computadores, nomeadamente ao nível do funcionamento do protocolo IP, da fragmentação de datagramas e do comportamento de ferramentas de diagnóstico como o traceroute e o ping.

Numa primeira fase, analisou-se detalhadamente o funcionamento dos datagramas IP e das mensagens ICMP, recorrendo ao Wireshark para observar o tráfego gerado. Este estudo permitiu compreender o processo de fragmentação de pacotes quando o tamanho do

datagrama excede o MTU da rede, bem como o papel de campos como o Identification, Fragment Offset e Flags na reconstrução do datagrama original. Para além disso, foi possível analisar o comportamento do TTL e o cálculo do RTT, consolidando a compreensão do percurso e do atraso dos pacotes na rede.

Numa segunda fase, exploraram-se conceitos de endereçamento e encaminhamento IP, incluindo a criação de sub-redes e a configuração de rotas estáticas. Através da implementação prática em topologias no CORE, foi possível identificar e corrigir problemas de encaminhamento, como loops e ausência de rotas de retorno, evidenciando a importância de tabelas de encaminhamento corretas para garantir a conectividade entre dispositivos.

Adicionalmente, foi estudado o impacto do princípio de Longest Prefix Match no processo de encaminhamento, bem como a diferença entre rotas mais específicas e rotas genéricas. Por fim, foi aplicada a técnica de sumarização de rotas (supernetting), permitindo reduzir o número de entradas na tabela de encaminhamento e melhorar a eficiência da rede, mantendo simultaneamente a conectividade.

Em suma, este trabalho permitiu consolidar conhecimentos teóricos e aplicá-los na prática, proporcionando uma compreensão mais aprofundada do diagnóstico, configuração e otimização de redes IP. A utilização de ferramentas como o Wireshark e comandos como ping e traceroute, aliada à simulação de cenários no CORE, revelou-se essencial para compreender o funcionamento real das redes de computadores e os desafios associados ao seu correto funcionamento.

TP2



Universidade do Minho
Escola de Engenharia

Redes de Computadores

RC-TP2-PL52

Hasti Abdollahvand - A102185
Sívio João Freitas Pontes - A100762
Hugo Soares - A107293

1. Introdução	4
2. Arquitetura do Sistema	4
2.1 Visão Geral	4
2.2 Estrutura do Programa	5
2.3 Fluxo de Processamento dos Pacotes	5
3. Captura e Parsing de Pacotes	6
3.1 Captura de Pacotes	6
3.2 Identificação da Camada de Ligação	6
3.3 Parsing dos Protocolos	7
4. Protocolos Suportados e Análise	7
4.1 ARP	7
4.2 ICMP	8
4.3 TCP	8
4.4 UDP	8
4.5 DNS	9
4.6 HTTP	9
4.7 DHCP	9
4.8 Considerações Finais	10
5. Filtros e Parâmetros de Execução	10
5.1 Parâmetros de Execução	10
5.2 Seleção da Interface de Rede	11
5.3 Filtro por Protocolo	11
5.4 Filtro por Endereço IP	12
5.5 Filtros por Endereço MAC	12
5.6 Filtros por Endereço BPF	12
5.7 Modos de Captura Avançados	13
5.8 Comparação entre Mecanismos de Filtragem	13
5.9 Considerações Finais	13
6. Sistema de Logging	13
6.1 Visão Geral	13
6.2 Formatos de saída	14
6.3 Estrutura de dados	15
6.4 Funcionamento do sistema Logging	15
6.5 Integração com o Modo de Execução	16
6.6 Vantagens do Sistema de Logging	16
6.7 Considerações Finais	16
7. Testes e Validação	16
7.1 Ambiente de Teste	16
7.2 Metodologia	16
7.3 Testes com ICMP	17

7.4 Testes com DNS	17
7.5 Testes com HTTP	17
7.6 Testes com Tráfego TCP/UDP	18
7.7 Validação com Wireshark	18
7.8 Considerações Finais	18
8. Execução no CORE	19
8.1 Topologia da Rede	19
8.2 Metodologia de Teste	19
8.3 Geração de Tráfego UDP	20
8.4 Observação de Mensagens ICMP	21
8.5 Observação de Tráfego ARP	21
8.6 Observação de Tráfego TCP	22
8.7 Validação com Wireshark	23
8.8 Considerações Finais	24
9. Limitações	24
9.1 Limitações na Análise de Protocolos Aplicacionais	24
9.2 Dependência de Privilégios de Sistema	25
9.3 Dependência da Interface de Rede	25
9.4 Limitações na Captura de Tráfego Específico	25
9.5 Suporte Limitado para Redes 802.11	25
10. Conclusão	26

1. Introdução

O presente trabalho, realizado no âmbito da unidade curricular de Redes de Computadores, tem como principal objetivo o desenvolvimento de um packet sniffer capaz de capturar, analisar e classificar tráfego de rede. Este tipo de ferramenta permite observar o funcionamento interno das comunicações em rede, contribuindo para uma melhor compreensão dos protocolos estudados nas aulas teóricas.

A aplicação desenvolvida foi implementada em linguagem Python, utilizando a biblioteca Scapy para a captura de pacotes, sendo o parsing e análise dos mesmos realizados manualmente através da interpretação dos bytes crus. O sistema é capaz de identificar diversos protocolos de rede, incluindo ARP, ICMP, TCP, UDP, DNS, DHCP, HTTP e IPv4, permitindo a extração de informação relevante de cada pacote.

O sniffer suporta diferentes modos de execução, incluindo visualização em tempo real (live) e armazenamento dos dados capturados em ficheiros, nos formatos TXT, CSV e JSON. Adicionalmente, foram implementados mecanismos de filtragem por protocolo, endereço IP, endereço MAC e expressões BPF, permitindo ao utilizador focar a análise em tipos específicos de tráfego.

O trabalho foi desenvolvido em duas fases distintas: numa primeira fase, a aplicação será testada num ambiente controlado utilizando o emulador CORE, permitindo a geração e análise de tráfego específico; numa segunda fase, o sniffer é executado sobre uma interface de rede real, possibilitando a observação de padrões de comunicação reais.

2. Arquitetura do Sistema

2.1 Visão Geral

O sistema desenvolvido consiste num packet sniffer implementado em Python, responsável pela captura, análise e apresentação de tráfego de rede. A arquitetura da aplicação segue um modelo sequencial composto por várias etapas, desde a captura dos pacotes até à sua apresentação ou armazenamento.

O funcionamento geral do sistema pode ser descrito pelas seguintes fases: captura de pacotes a partir de uma interface de rede, identificação do tipo de enlace (link layer), parsing dos protocolos presentes em cada pacote, aplicação de filtros definidos pelo utilizador e, por fim, apresentação dos resultados em tempo real ou armazenamento em ficheiros.

A captura de pacotes é realizada recorrendo à biblioteca Scapy, que permite a obtenção de dados brutos diretamente da interface de rede. A partir destes dados, o programa realiza manualmente o processamento das diferentes camadas do modelo de comunicação, garantindo maior controlo sobre a interpretação dos protocolos.

2.2 Estrutura do Programa

O sniffer foi desenvolvido num único ficheiro principal, **sniffer.py**, no qual se encontram organizadas as diferentes funcionalidades do sistema. A estrutura do programa encontra-se dividida em módulos funcionais, responsáveis por diferentes etapas do processamento dos pacotes.

A captura de pacotes é realizada através da função **sniff_scapy**, que utiliza a biblioteca Scapy para recolher os pacotes da interface de rede selecionada. Cada pacote capturado é convertido para uma sequência de bytes e encaminhado para o sistema de parsing.

O processamento dos pacotes é centralizado na função **parse_packet**, responsável por identificar o tipo de trama e encaminhar o pacote para as funções específicas de parsing. O sistema inclui funções dedicadas à análise de diferentes protocolos, tais como **parse_arp**, **parse_ipv4**, **parse_icmp**, **parse_tcp**, **parse_udp**, **parse_dns**, **parse_dhcp** e **parse_http**.

Para além disso, existe uma função responsável pelo processamento de pacotes IPv4 (**process_ipv4**), que coordena a identificação dos protocolos de camada superior (ICMP, TCP e UDP) e invoca os respetivos módulos de parsing.

O sistema inclui ainda um módulo de filtragem (**apply_filters**), que permite restringir os pacotes apresentados com base em critérios definidos pelo utilizador, e um módulo de logging, responsável pelo armazenamento dos dados capturados em diferentes formatos.

2.3 Fluxo de Processamento dos Pacotes

O processamento de cada pacote segue uma sequência bem definida de etapas, desde a sua captura até à sua apresentação final. O fluxo de execução pode ser descrito da seguinte forma:

1. **Captura do pacote**
O pacote é capturado através da biblioteca Scapy e convertido numa sequência de bytes.
2. **Identificação do tipo de enlace**
O sistema analisa os primeiros bytes do pacote para determinar o tipo de trama, podendo identificar Ethernet, tráfego IPv4 direto ou outros formatos suportados.
3. **Parsing da camada de rede**
Caso o pacote corresponda a IPv4, são extraídos os endereços IP de origem e destino, bem como o protocolo da camada superior.
4. **Parsing da camada de transporte**
Dependendo do protocolo identificado (ICMP, TCP ou UDP), o sistema invoca a função correspondente para analisar os campos específicos.
5. **Análise de protocolos aplicacionais**
Para protocolos como DNS, DHCP e HTTP, o sistema realiza uma análise adicional do

conteúdo, permitindo extrair informação relevante, como nomes de domínio ou métodos HTTP.

6. **Aplicação de filtros**

Após o parsing, os pacotes são filtrados com base nos critérios definidos pelo utilizador, como protocolo, endereço IP ou endereço MAC.

7. **Apresentação e logging**

Os pacotes que passam os filtros são apresentados em tempo real na consola e/ou armazenados em ficheiros, conforme os parâmetros definidos.

Este fluxo modular permite uma separação clara das diferentes responsabilidades do sistema, facilitando a sua manutenção, extensão e compreensão.

3. Captura e Parsing de Pacotes

3.1 Captura de Pacotes

A captura de pacotes foi realizada utilizando a biblioteca **Scapy**, que permite aceder diretamente ao tráfego de rede através da interface selecionada. Esta abordagem simplifica o processo de captura, garantindo ao mesmo tempo flexibilidade na manipulação dos dados obtidos.

Durante os testes em ambiente real, foi utilizada a interface de rede **en0**, correspondente à interface principal do sistema macOS. A execução do sniffer nesta interface permitiu capturar tráfego real gerado pelo sistema, incluindo comunicações de diferentes protocolos.

A captura é realizada através da função **sniff_scapy**, que invoca o método **sniff()** da biblioteca Scapy, permitindo a recolha contínua de pacotes e o seu encaminhamento para o sistema de processamento.

3.2 Identificação da Camada de Ligação

Após a captura, o sistema procede à identificação do tipo de enlace (link layer) presente no pacote. Esta etapa é fundamental para garantir que o parsing subsequente é realizado corretamente, uma vez que diferentes tipos de tramas possuem estruturas distintas.

O sniffer implementa um mecanismo de deteção automática, capaz de identificar vários formatos de pacotes, incluindo:

- Tramas Ethernet, através da análise do campo ethertype;
- Pacotes IPv4 diretos (raw IPv4);
- Formatos intermédios como Linux Cooked Capture (SLL).

Esta abordagem permite ao sistema adaptar-se a diferentes ambientes de execução, garantindo robustez e compatibilidade com múltiplos tipos de interfaces.

3.3 Parsing dos Protocolos

O parsing dos pacotes é realizado manualmente através da análise dos bytes capturados, recorrendo ao módulo **struct** da linguagem Python para extrair os diferentes campos dos cabeçalhos.

Após a identificação da camada de ligação, o sistema procede à análise da camada de rede, extraindo informações como os endereços IP de origem e destino, bem como o protocolo da camada superior.

Para pacotes IPv4, o sistema identifica o protocolo de transporte associado e invoca as funções correspondentes:

- ICMP: análise de mensagens de controlo, como Echo Request e Echo Reply;
- TCP: extração de portas, flags (SYN, ACK, PSH, entre outras) e tamanho da janela;
- UDP: identificação de portas e encaminhamento para protocolos de aplicação.

Nos casos em que o tráfego corresponde a protocolos de camada aplicacional, como DNS, DHCP ou HTTP, o sistema realiza uma análise adicional do payload, permitindo extrair informação relevante, como nomes de domínio, tipos de mensagens DHCP ou métodos HTTP.

Esta abordagem de parsing manual permite um maior controlo sobre a análise dos pacotes e garante uma compreensão aprofundada da estrutura interna dos diferentes protocolos.

4. Protocolos Suportados e Análise

O sniffer desenvolvido é capaz de identificar e analisar diversos protocolos pertencentes às diferentes camadas da pilha de comunicação. Nesta secção são descritos os principais protocolos suportados, bem como os resultados obtidos durante os testes realizados em ambiente real e no CORE.

4.1 ARP

O protocolo ARP é utilizado para mapear endereços IP em endereços MAC dentro de uma rede local. Sempre que um host pretende comunicar com outro na mesma sub-rede, necessita de conhecer o endereço MAC correspondente ao endereço IP de destino.

Durante os testes no ambiente CORE, foram observadas mensagens ARP do tipo request e reply, como por exemplo:

```
Who has 10.0.1.20? Tell 10.0.1.1  
10.0.1.20 is at 00:00:00:aa:00:03
```

Estas mensagens mostram o processo de resolução de endereços, no qual um host envia um pedido (ARP Request) e o destinatário responde com o seu endereço físico (ARP Reply).

A correta identificação deste protocolo pelo sniffer demonstra a capacidade de análise ao nível da camada de enlace.

4.2 ICMP

O protocolo ICMP é utilizado para enviar mensagens de controlo e erro entre dispositivos de rede.

Durante os testes em ambiente real, foram geradas mensagens ICMP através do comando: **ping 8.8.8.8**

O sniffer capturou corretamente mensagens do tipo:

- Echo Request
- Echo Reply

Para além disso, nos testes realizados no CORE, foram observadas mensagens ICMP do tipo: Destination Unreachable (Port Unreachable)

Estas mensagens surgem como resposta ao envio de pacotes para portas não disponíveis, evidenciando o papel do ICMP na sinalização de erros na rede.

4.3 TCP

O protocolo TCP é um protocolo orientado à ligação, responsável por garantir a entrega fiável dos dados entre origem e destino.

Durante os testes, foram observadas tentativas de estabelecimento de ligação TCP, caracterizadas pelo envio de pacotes com a flag: **SYN** e respostas do tipo: **RST, ACK**

Este comportamento indica que a ligação foi recusada pelo destino, por não existir um serviço ativo na porta especificada.

O sniffer demonstrou capacidade para identificar corretamente as flags TCP, permitindo analisar o estado das ligações.

4.4 UDP

O protocolo UDP é um protocolo não orientado à ligação, caracterizado pela ausência de mecanismos de controlo de entrega.

Durante os testes no CORE, o tráfego UDP foi gerado utilizando a ferramenta netcat, com comandos do tipo:

echo "Hello" | nc -u 10.0.1.20 1234

O sniffer capturou corretamente os pacotes UDP, identificando:

- endereços IP de origem e destino;
- portas utilizadas;
- tamanho dos pacotes.

A ausência de confirmação de entrega neste protocolo foi evidenciada pela geração de mensagens ICMP em resposta a portas não abertas.

4.5 DNS

O protocolo DNS é responsável pela resolução de nomes de domínio em endereços IP. Durante os testes em ambiente real, foram geradas consultas DNS através do comando: **nslookup google.com**

O sniffer capturou mensagens DNS do tipo:

- query (pedido de resolução);
- response (resposta com endereços IP).

Foi possível observar respostas contendo múltiplos endereços IP para o mesmo domínio, evidenciando mecanismos como balanceamento de carga.

Estes resultados demonstram a capacidade do sniffer para interpretar corretamente o protocolo DNS e extrair informação relevante.

4.6 HTTP

O protocolo HTTP opera ao nível da camada de aplicação, sendo utilizado para a comunicação entre clientes e servidores web.

Durante os testes, foi capturado tráfego HTTP correspondente a pedidos e respostas, incluindo: **GET / HTTP/1.1** e **HTTP/1.1 304 Not Modified**

O sniffer foi capaz de identificar corretamente o protocolo HTTP com base nas portas utilizadas (80 e 8080) e analisar o conteúdo do payload.

Importa referir que o tráfego HTTPS não foi analisado ao nível do conteúdo, uma vez que se encontra cifrado, impedindo a interpretação dos dados da camada de aplicação.

4.7 DHCP

O protocolo DHCP é responsável pela atribuição automática de endereços IP e outros parâmetros de rede aos dispositivos de uma rede.

O sniffer desenvolvido inclui suporte para a identificação e parsing de mensagens DHCP, permitindo reconhecer diferentes tipos de mensagens, como:

- Discover

- Offer
- Request
- Acknowledgement (ACK)

Estas mensagens fazem parte do processo de configuração dinâmica de um host numa rede.

No entanto, durante os testes realizados, não foi possível observar tráfego DHCP de forma consistente. Isto deve-se ao facto de este protocolo ser tipicamente utilizado apenas no momento em que um dispositivo se liga à rede ou renova o seu endereço IP, não estando presente em comunicações regulares.

Apesar disso, a funcionalidade encontra-se implementada no sistema, estando preparada para analisar este tipo de tráfego quando presente.

4.8 Considerações Finais

A análise dos diferentes protocolos demonstrou que o sniffer desenvolvido é capaz de identificar corretamente tráfego pertencente a várias camadas da pilha de comunicação. Os testes realizados, tanto em ambiente real como no CORE, permitiram observar diferentes tipos de comunicação, incluindo resolução de endereços (ARP), controlo e erros (ICMP), transporte de dados (TCP e UDP) e protocolos de aplicação (DNS e HTTP). Esta diversidade de cenários confirma a robustez da aplicação e a sua capacidade de análise de tráfego em diferentes contextos.

5. Filtros e Parâmetros de Execução

Os mecanismos de filtragem implementados permitem ao utilizador controlar de forma eficaz o tráfego analisado, facilitando a identificação de padrões específicos e a análise de protocolos de interesse. Os testes realizados confirmam o correto funcionamento dos filtros por protocolo e endereço IP, demonstrando a robustez do sistema.

5.1 Parâmetros de Execução

O sniffer desenvolvido disponibiliza um conjunto abrangente de parâmetros de linha de comandos que permitem configurar o comportamento da aplicação e adaptar a captura de tráfego a diferentes cenários. Estes parâmetros permitem controlar a interface de rede, o número de pacotes capturados, os modos de visualização, o logging e os mecanismos de filtragem.

Os principais parâmetros suportados são:

- **-i** ou **--i**: define a interface de rede onde será efetuada a captura;
- **-c** ou **--count**: limita o número de pacotes capturados;1
- **--live**: apresenta os pacotes em tempo real na consola;
- **--no-live**: desativa a apresentação em consola;
- **--log-txt**, **--log-csv**, **--log-json**: guardam os resultados em ficheiros;

- **--filter-proto**: filtra por protocolo;
- **--filter-ip**: filtra por endereço IP;
- **--filter-mac**: filtra por endereço MAC;
- **--bpf**: aplica uma expressão BPF ao nível da captura.
- **--list-ifaces**: lista todas as interfaces de rede disponíveis no sistema;
- **--dot11**: ativa o suporte para captura de tramas IEEE 802.11 (Wi-Fi);
- **--raw-socket**: ativa um modo alternativo de captura baseado em raw sockets.

Caso o utilizador não especifique nenhum modo de saída (visualização ou logging), o sistema ativa automaticamente o modo de visualização em tempo real (live), garantindo que os pacotes capturados são apresentados ao utilizador.

5.2 Seleção da Interface de Rede

A escolha da interface de rede é essencial para o correto funcionamento do sniffer. Esta pode ser especificada diretamente através do parâmetro **-i**.

Durante os testes realizados em ambiente real, foi utilizada a interface **en0**, correspondente à interface principal do sistema macOS.

Caso o utilizador não indique explicitamente uma interface, o sistema entra em modo interativo: apresenta uma lista das interfaces disponíveis e solicita ao utilizador que selecione uma delas. Este comportamento permite evitar erros na escolha da interface, especialmente em sistemas com múltiplas interfaces ativas.

Adicionalmente, o parâmetro **--list-ifaces** permite listar previamente todas as interfaces disponíveis, possibilitando ao utilizador identificar a interface correta antes de iniciar a captura.

5.3 Filtro por Protocolo

O sniffer permite filtrar pacotes com base no protocolo identificado, através do parâmetro:

--filter-proto <PROTOCOLO>

Este filtro é aplicado após o parsing dos pacotes, atuando ao nível da aplicação.

Foram realizados testes com diferentes protocolos. No caso do ICMP, a execução do sniffer com o respetivo filtro permitiu observar exclusivamente mensagens do tipo Echo Request e Echo Reply, geradas através do comando **ping**.

```
sudo python3 sniffer.py -i en0 --filter-proto ICMP
```

De forma semelhante, ao aplicar um filtro para DNS e gerar tráfego com **nslookup**, o sistema apresentou apenas pacotes DNS, incluindo queries e responses. Estes resultados demonstram que o filtro por protocolo funciona corretamente e permite isolar tráfego específico.

```
sudo python3 sniffer.py -i en0 --filter-proto DNS
```

5.4 Filtro por Endereço IP

O sniffer permite também a filtragem de pacotes com base no endereço IP de origem ou destino, através do parâmetro:

```
--filter-ip <ENDEREÇO_IP>
```

Para validar esta funcionalidade, foi realizado um teste utilizando o endereço IP 8.8.8.8:

```
sudo python3 sniffer.py -i en0 --filter-ip 8.8.8.8
```

Durante o teste, foram gerados pacotes ICMP com o comando ping. O sniffer apresentou apenas pacotes associados ao endereço especificado, demonstrando que o filtro funciona corretamente tanto para tráfego de origem como de destino.

5.5 Filtros por Endereço MAC

O sistema inclui ainda a possibilidade de filtrar pacotes com base no endereço MAC:

```
--filter-mac <ENDEREÇO_MAC>
```

Este tipo de filtro permite restringir a análise a dispositivos específicos dentro da rede local, sendo particularmente útil em cenários onde existem múltiplos hosts a gerar tráfego simultaneamente.

Embora este filtro não tenha sido extensivamente testado durante os ensaios realizados, a sua implementação encontra-se funcional e integrada no sistema de filtragem.

5.6 Filtros por Endereço BPF

Para além dos filtros implementados ao nível da aplicação, o sniffer suporta expressões BPF (Berkeley Packet Filter), que permitem filtrar o tráfego diretamente durante a fase de captura. A utilização deste filtro é feita através do parâmetro:

```
--bpf "<expressão>"
```

As expressões BPF são aplicadas pela biblioteca de captura (libpcap, através do Scapy), sendo executadas antes dos pacotes serem entregues à aplicação. Isto significa que apenas os pacotes que satisfazem a condição definida são efetivamente capturados.

Exemplos de utilização incluem:

```
sudo python3 sniffer.py --bpf "tcp port 80"  
sudo python3 sniffer.py --bpf "udp port 53"  
sudo python3 sniffer.py --bpf "icmp"  
sudo python3 sniffer.py --bpf "arp"
```

A principal vantagem deste tipo de filtro é a sua eficiência, uma vez que reduz o número de pacotes processados pela aplicação. No entanto, apresenta menor flexibilidade em

comparação com os filtros internos, que operam após o parsing e permitem decisões mais detalhadas.

5.7 Modos de Captura Avançados

O sniffer inclui ainda funcionalidades adicionais que permitem explorar diferentes modos de captura.

O parâmetro **--dot11** ativa o parsing de tramas IEEE 802.11, permitindo analisar diretamente tráfego Wi-Fi ao nível da camada de enlace. Esta funcionalidade depende do suporte da interface de rede e da ativação do modo monitor, não tendo sido utilizada extensivamente durante os testes realizados.

O parâmetro **--raw-socket** permite utilizar um mecanismo alternativo de captura baseado em raw sockets, possibilitando a obtenção direta dos pacotes ao nível do sistema operativo. Apesar desta funcionalidade oferecer maior controlo, na prática foi privilegiada a utilização da biblioteca Scapy, devido à sua simplicidade e portabilidade.

5.8 Comparação entre Mecanismos de Filtragem

Os filtros implementados podem ser divididos em duas categorias principais:

- Filtros BPF: aplicados durante a captura, antes dos pacotes chegarem à aplicação;
- Filtros internos (**--filter-proto**, **--filter-ip**, **--filter-mac**): aplicados após o parsing dos pacotes.

Esta distinção é importante, pois os filtros BPF permitem maior eficiência ao reduzir o volume de dados processados, enquanto os filtros internos permitem uma análise mais detalhada baseada na informação já interpretada pelo sniffer.

5.9 Considerações Finais

Os mecanismos de filtragem e os parâmetros de execução implementados conferem ao sniffer um elevado grau de flexibilidade e controlo. A possibilidade de seleccionar a interface, definir filtros e escolher o modo de execução permite adaptar a aplicação a diferentes cenários de análise.

Destaca-se o comportamento adotado quando a interface não é especificada, no qual o sistema solicita explicitamente ao utilizador a escolha da interface, garantindo uma utilização mais segura e controlada.

6. Sistema de Logging

6.1 Visão Geral

O sniffer desenvolvido integra um sistema de logging que permite o armazenamento dos pacotes capturados em diferentes formatos, possibilitando a análise posterior dos dados. Este

mecanismo é essencial para cenários em que não é suficiente observar o tráfego em tempo real, sendo necessário guardar os resultados para estudo ou processamento adicional. A aplicação suporta três formatos distintos de saída: TXT, CSV e JSON, permitindo ao utilizador escolher o formato mais adequado ao contexto de utilização.

6.2 Formatos de saída

O sistema de logging suporta três formatos distintos de saída: TXT, CSV e JSON, cada um com características próprias e adequados a diferentes tipos de análise.

- **TXT**: formato textual simples, no qual cada pacote é representado numa linha com os campos separados por delimitadores (|).

Este formato é orientado para leitura humana e análise direta no terminal ou em ficheiros de texto.

Exemplo:

```
1 | 2026-04-30 16:37:10.400 | 0.019521 | en0 | UDP | 3c:06:30:5a:74:10 | 32:cb:2b:02:ef:94 | 10
```

- **CSV**: formato estruturado em colunas separadas por vírgulas, adequado para importação em ferramentas como Excel, LibreOffice Calc ou scripts de análise de dados.

Exemplo:

```
no,ts_abs,ts_rel,iface,proto,src_mac,dst_mac,src_ip,dst_ip,size,summary
1,2026-04-30
16:39:15.585,0.013525,en0,UDP,3c:06:30:5a:74:10,32:cb:2b:02:ef:94,10.79.32.80,104.29.158.1
6,146,UDP 10.79.32.80:56218 → 104.29.158.16:19302 (104B)
```

- **JSON**: formato estruturado baseado em objetos, onde cada pacote é representado como um dicionário dentro de uma lista. Este formato é especialmente útil para integração com aplicações ou processamento programático.

Exemplo:

```
[
  {
    "no": 1,
    "ts_abs": "2026-04-30 16:39:31.926",
    "ts_rel": "0.008771",
    "iface": "en0",
    "proto": "UDP",
    "src_mac": "3c:06:30:5a:74:10",
    "dst_mac": "32:cb:2b:02:ef:94",
    "src_ip": "10.79.32.80",
    "dst_ip": "104.29.158.16",
    "size": 146,
```



```
"summary": "UDP 10.79.32.80:56590 → 104.29.158.16:19302 (104B)"
}
]
```

A seleção do formato é feita através dos parâmetros:

--log-txt <ficheiro>

--log-csv <ficheiro>

--log-json <ficheiro>

Sendo possível utilizar vários formatos em simultâneo durante a mesma execução.

6.3 Estrutura de dados

Independentemente do formato de saída, o sistema mantém uma estrutura consistente de dados, garantindo uniformidade na informação registada.

Cada pacote capturado é representado através dos seguintes campos:

- **no**: número sequencial do pacote;
- **ts_abs**: timestamp absoluto (data e hora da captura);
- **ts_rel**: tempo relativo desde o início da captura;
- **iface**: interface de rede utilizada;
- **proto**: protocolo identificado;
- **src_mac** e **dst_mac**: endereços MAC de origem e destino;
- **src_ip** e **dst_ip**: endereços IP de origem e destino;
- **size**: tamanho total do pacote em bytes;
- **summary**: descrição textual resumida do pacote.

A presença destes campos permite não só identificar os participantes da comunicação, mas também analisar o comportamento temporal e o tipo de tráfego observado.

6.4 Funcionamento do sistema Logging

O logging é realizado de forma integrada com o processo de captura e parsing dos pacotes. Após o processamento de cada pacote, os dados são convertidos para um formato estruturado e registados nos ficheiros definidos pelo utilizador.

No caso dos formatos TXT e CSV, os dados são escritos imediatamente após o processamento de cada pacote, garantindo que a informação é persistida de forma incremental. Já no formato JSON, os dados são acumulados em memória e escritos no final da execução, sob a forma de uma lista de registos.

Este comportamento permite otimizar a escrita em JSON, evitando múltiplas operações de escrita e garantindo a consistência do ficheiro final.

6.5 Integração com o Modo de Execução

O sistema de logging pode ser utilizado em conjunto com o modo de visualização em tempo real (live) ou de forma independente. A aplicação permite, por exemplo:

- visualizar os pacotes e simultaneamente guardá-los em ficheiro;
- desativar a visualização (**--no-live**) e utilizar apenas o logging;
- utilizar múltiplos formatos de logging na mesma execução.

Caso o utilizador não especifique nenhum modo de saída, o sistema ativa automaticamente o modo live, garantindo que os dados capturados são apresentados na consola.

6.6 Vantagens do Sistema de Logging

O sistema de logging implementado constitui uma componente fundamental do sniffer, permitindo não apenas a observação em tempo real, mas também o armazenamento estruturado do tráfego capturado. A possibilidade de utilizar múltiplos formatos em simultâneo aumenta a versatilidade da aplicação e contribui para uma análise mais completa e rigorosa dos dados.

6.7 Considerações Finais

A implementação de múltiplos formatos de logging oferece diversas vantagens:

- permite a análise posterior dos dados capturados;
- facilita a integração com ferramentas externas de análise;
- possibilita a exportação de dados estruturados;
- melhora a reprodutibilidade dos testes realizados.

O formato CSV destaca-se pela sua simplicidade e compatibilidade com ferramentas de análise de dados, enquanto o JSON oferece maior flexibilidade para processamento programático.

7. Testes e Validação

7.1 Ambiente de Teste

Os testes do sniffer foram realizados em ambiente real, utilizando um sistema operativo macOS. A captura de pacotes foi efetuada através da interface de rede **en0**, correspondente à interface principal de ligação à rede (Wi-Fi).

Este ambiente permitiu observar tráfego real gerado pelo sistema, incluindo comunicações com servidores externos, resolução de nomes de domínio e pedidos HTTP.

7.2 Metodologia

A validação do funcionamento do sniffer foi realizada através da geração controlada de diferentes tipos de tráfego de rede, utilizando ferramentas standard do sistema operativo.

Em paralelo, foi utilizado o Wireshark como ferramenta de referência, permitindo comparar os resultados obtidos com o sniffer desenvolvido.

Os testes foram organizados por tipo de protocolo, de forma a validar a correta detecção e interpretação dos pacotes capturados.

7.3 Testes com ICMP

Para gerar tráfego ICMP, foi utilizado o comando:

ping 8.8.8.8 -c 4

Durante a execução, o sniffer capturou pacotes do tipo Echo Request e Echo Reply, correspondentes ao envio e recepção de mensagens ICMP.

Os resultados observados confirmam que o sistema identifica corretamente este protocolo, sendo possível acompanhar a comunicação entre o host local e o servidor remoto.

7.4 Testes com DNS

Para validar o funcionamento do protocolo DNS, foi utilizado o comando:

nslookup google.com

A execução deste comando gerou tráfego DNS, que foi capturado pelo sniffer sob a forma de mensagens de query e response. Foi possível observar pedidos de resolução de nomes e as respectivas respostas contendo os endereços IP associados ao domínio consultado.

A análise dos resultados confirma que o sniffer identifica corretamente o protocolo DNS, bem como os campos relevantes associados às mensagens de resolução de nomes.

7.5 Testes com HTTP

Para gerar tráfego HTTP, foi acessado um website que utiliza comunicação não cifrada (HTTP), permitindo observar diretamente os dados da camada de aplicação.

Como se pode observar na Figura X, o sniffer capturou um pedido HTTP do tipo:

GET / HTTP/1.1

bem como a respectiva resposta:

HTTP/1.1 304 Not Modified

Estes resultados demonstram que o sistema consegue identificar e interpretar corretamente o protocolo HTTP, incluindo a extração de informação relevante do payload.

Importa referir que o tráfego HTTPS não foi analisado ao nível do conteúdo, uma vez que este se encontra cifrado, impossibilitando a inspeção direta dos dados da camada de aplicação.

7.6 Testes com Tráfego TCP/UDP

Para além dos protocolos específicos testados, foi também observada comunicação geral baseada em TCP e UDP, resultante da utilização normal do sistema (acesso à web, serviços em background, etc.).

O sniffer demonstrou capacidade para identificar corretamente estes protocolos, apresentando informação relevante como endereços IP, portas de origem e destino, e flags TCP.

7.7 Validação com Wireshark

Para garantir a fiabilidade dos resultados, os dados capturados pelo sniffer foram comparados com os obtidos através da ferramenta Wireshark.

Como se pode observar nas Figuras X e Y, os pacotes identificados pelo sniffer correspondem aos observados no Wireshark, tanto ao nível dos protocolos como da informação associada (endereços IP, tipo de mensagem e sequência de comunicação).

Esta comparação permitiu validar a correta implementação do sistema de captura e parsing de pacotes.

7.8 Considerações Finais

Os testes realizados em ambiente real demonstram que o sniffer desenvolvido é capaz de capturar e interpretar corretamente diferentes tipos de tráfego de rede, incluindo ICMP, DNS, HTTP, TCP e UDP.

A utilização de uma interface real (**en0**) e a validação com Wireshark reforçam a fiabilidade dos resultados obtidos, evidenciando o correto funcionamento da aplicação em cenários reais.

8. Execução no CORE

8.1 Topologia da Rede

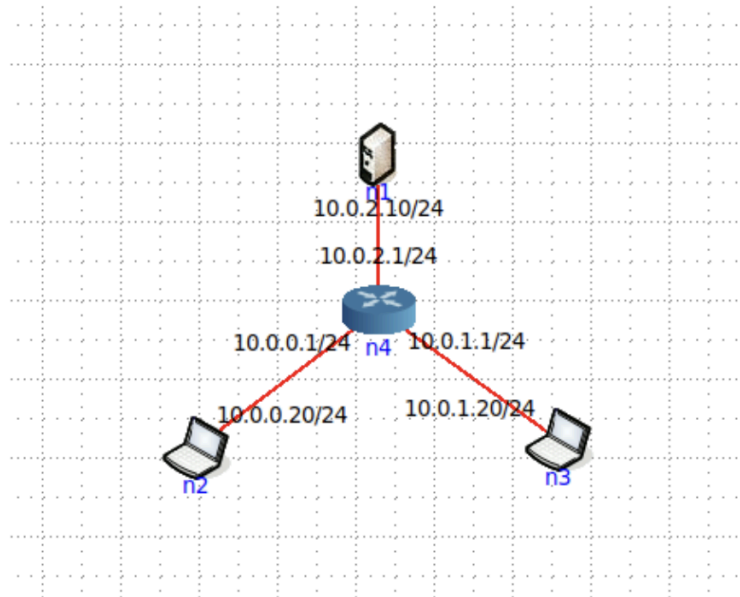


Figura 1 — Topologia da rede configurada no CORE, composta por três hosts e um router, organizados em três sub-redes distintas.

Para complementar os testes realizados em ambiente real, foi utilizada a ferramenta CORE (Common Open Research Emulator), permitindo a criação de um cenário de rede controlado.

A topologia configurada é composta por:

- um router central (**n4**);
- três hosts (**n1**, **n2** e **n3**);
- três sub-redes distintas:
 - 10.0.0.0/24
 - 10.0.1.0/24
 - 10.0.2.0/24

Cada host encontra-se ligado ao router através de uma interface própria, sendo o encaminhamento entre redes realizado pelo router. Esta configuração permite simular comunicações entre diferentes sub-redes, gerando tráfego mais diversificado do que num ambiente simples.

8.2 Metodologia de Teste

Os testes no ambiente CORE foram realizados através da execução de comandos diretamente nos terminais dos nós da rede, com o objetivo de gerar diferentes tipos de tráfego.

Em particular, foram utilizados:

- envio de pacotes UDP com **netcat**;
- tentativas de comunicação TCP para portas sem serviço ativo;
- geração indireta de mensagens ICMP associadas a erros de comunicação;
- comunicação entre hosts pertencentes a diferentes sub-redes.

Em paralelo, o tráfego foi observado através do sniffer desenvolvido e comparado com os resultados obtidos no Wireshark, garantindo a validação dos dados capturados.

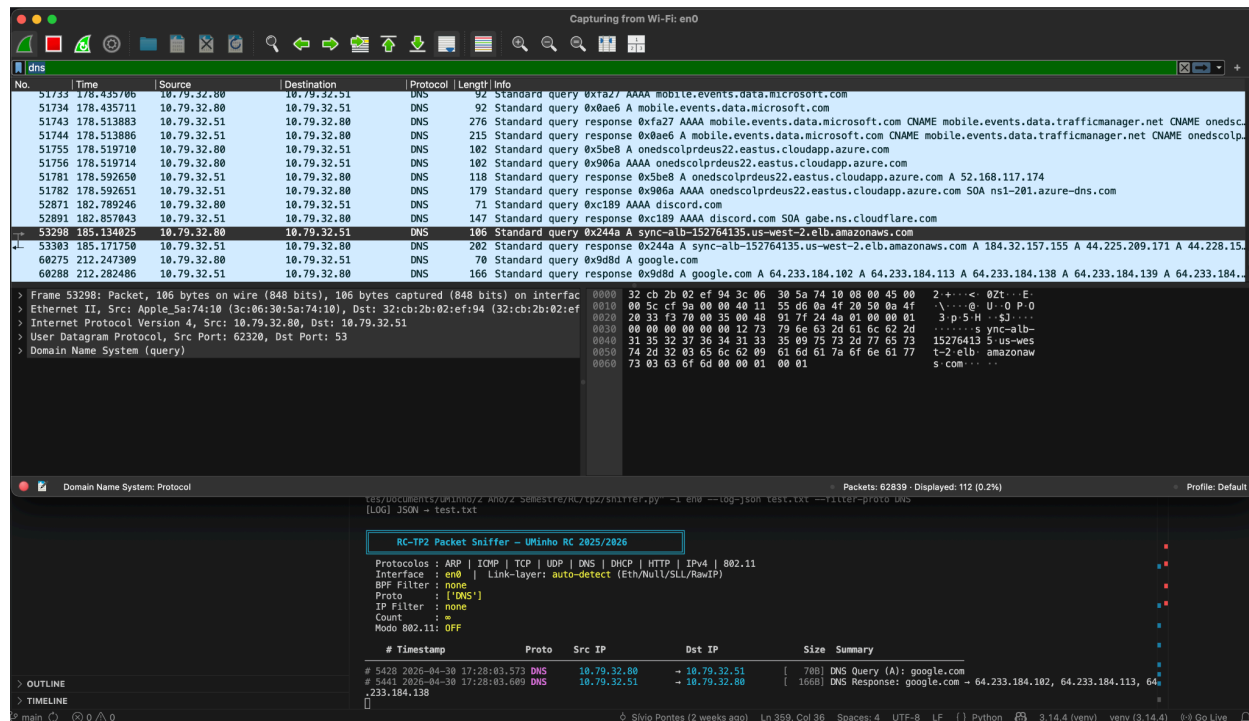


Figura 2 — Captura de tráfego DNS evidenciando uma query para o domínio google.com e a respectiva resposta contendo os endereços IP associados.

8.3 Geração de Tráfego UDP

O tráfego UDP foi gerado a partir de um dos hosts utilizando o comando:

echo "Hello" | nc -u 10.0.1.20 1234

Este comando envia uma mensagem UDP para o host de destino, utilizando a porta 1234.

Durante a captura, foram observados pacotes UDP entre os hosts, contendo informação como:

- endereço IP de origem e destino;
- portas utilizadas;
- tamanho do payload.

Este comportamento confirma o correto funcionamento do protocolo UDP e a capacidade do sniffer em identificar este tipo de tráfego.

8.4 Observação de Mensagens ICMP

Como consequência do envio de pacotes UDP para uma porta sem serviço ativo no destino, foram geradas mensagens ICMP do tipo:

ICMP Destination Unreachable (Port Unreachable)

Estas mensagens indicam que o destino foi alcançado, mas a porta especificada não se encontra disponível. Este comportamento foi claramente observado tanto no Wireshark como no sniffer desenvolvido.

A presença destas mensagens demonstra o papel do protocolo ICMP na sinalização de erros na rede.

8.5 Observação de Tráfego ARP

Durante os testes, foram também observadas mensagens ARP, necessárias para a resolução de endereços IP em endereços MAC dentro da rede local.

Exemplos observados incluem:

ARP Request: Who has 10.0.1.20? Tell 10.0.1.1

ARP Reply: 10.0.1.20 is at 00:00:00:aa:00:03

Este processo ocorre sempre que um host necessita de comunicar com outro e não possui ainda o respetivo endereço MAC.

A observação destas mensagens confirma o correto funcionamento da comunicação ao nível da camada de enlace.

The screenshot displays a Wireshark packet capture from the veth3.0.9 interface. The packet list shows various ICMP and ARP messages. The packet details pane highlights an ICMP Destination Unreachable (Port Unreachable) message. The packet bytes pane shows the raw data of the selected packet. To the right, a terminal window shows the execution of a netcat listener on port 1234, which receives a connection from 10.0.1.20.

No.	Time	Source	Destination	Protocol	Length	Info
15	225.280202030	fe80::200:ff:feaa:3	ff02::2	ICMPv6	70	Router Solicitation from 00:00:00:aa:00:03
16	257.223948244	00:00:00:aa:00:02	Broadcast	ARP	42	Who has 10.0.1.20? Tell 10.0.1.1
17	257.224250235	00:00:00:aa:00:03	00:00:00:aa:00:02	ARP	42	10.0.1.20 is at 00:00:00:aa:00:03
18	257.224369243	10.0.0.20	10.0.1.20	UDP	48	60957 -> 1234 Len=6
19	257.224624022	10.0.1.20	10.0.1.20	ICMP	76	Destination unreachable (Port unreachable)
20	262.400461166	00:00:00:aa:00:03	00:00:00:aa:00:02	ARP	42	Who has 10.0.1.1? Tell 10.0.1.20
21	262.405137972	00:00:00:aa:00:02	00:00:00:aa:00:03	ARP	42	10.0.1.1 is at 00:00:00:aa:00:03
22	279.525162297	10.0.0.20	10.0.1.20	UDP	48	47030 -> 1234 Len=6
23	279.525551449	10.0.1.20	10.0.1.20	ICMP	76	Destination unreachable (Port unreachable)
24	284.612336614	00:00:00:aa:00:02	00:00:00:aa:00:03	ARP	42	Who has 10.0.1.20? Tell 10.0.1.1
25	284.612629065	00:00:00:aa:00:03	00:00:00:aa:00:02	ARP	42	10.0.1.20 is at 00:00:00:aa:00:03

```
4 2026-04-30 17:02:17.576 0x0600 N/A + N/A 1078 Ethertype 0x0600
5 2026-04-30 17:02:23.883 0x0600 N/A + N/A 708 Ethertype 0x0600
6 2026-04-30 17:03:49.878 0x0600 N/A + N/A 708 Ethertype 0x0600
7 2026-04-30 17:04:06.250 0x0600 N/A + N/A 708 Ethertype 0x0600
8 2026-04-30 17:04:23.637 0x0600 N/A + N/A 708 Ethertype 0x0600
9 2026-04-30 17:04:25.590 0x0600 N/A + N/A 1078 Ethertype 0x0600
10 2026-04-30 17:04:49.269 0x0600 N/A + N/A 708 Ethertype 0x0600
11 2026-04-30 17:05:21.217 ARP 10.0.1.1 + 10.0.1.20 428 ARP Request: who has 10.0.1.20? Tell 10.0.1.1
12 2026-04-30 17:05:21.229 ARP 10.0.1.20 + 10.0.1.1 428 ARP Reply: 10.0.1.20 is at 00:00:00:aa:00:03
13 2026-04-30 17:05:21.250 UDP 10.0.0.20 + 10.0.1.20:1234 (6B) 488 UDP 10.0.0.20:60957 -> 10.0.1.20:1234 (6B)
14 2026-04-30 17:05:21.265 ICMP 10.0.1.20 + 10.0.0.20 768 ICMP Destination Unreachable (Port Unreachable)
15 2026-04-30 17:05:26.387 ARP 10.0.1.20 + 10.0.1.1 428 ARP Request: who has 10.0.1.1? Tell 10.0.1.20
16 2026-04-30 17:05:26.392 ARP 10.0.1.1 + 10.0.1.20 428 ARP Reply: 10.0.1.1 is at 00:00:00:aa:00:02
17 2026-04-30 17:05:43.521 UDP 10.0.0.20 + 10.0.1.20 488 UDP 10.0.0.20:47030 -> 10.0.1.20:1234 (6B)
18 2026-04-30 17:05:43.554 ICMP 10.0.1.20 + 10.0.0.20 768 ICMP Destination Unreachable (Port Unreachable)
19 2026-04-30 17:05:48.583 ARP 10.0.1.1 + 10.0.1.20 428 ARP Request: who has 10.0.1.20? Tell 10.0.1.1
20 2026-04-30 17:05:48.577 ARP 10.0.1.20 + 10.0.1.1 428 ARP Reply: 10.0.1.20 is at 00:00:00:aa:00:03
```

Figura 3 — Captura de tráfego no ambiente CORE evidenciando mensagens ARP, pacotes UDP gerados com netcat e respostas ICMP do tipo “Destination Unreachable”, validando o funcionamento do sniffer.

8.6 Observação de Tráfego TCP

Foram também observadas tentativas de comunicação TCP entre hosts, nomeadamente através do envio de pacotes para portas onde não existia nenhum serviço ativo.

Nestes casos, foram capturados pacotes com flags características:

[SYN]

[RST, ACK]

O pacote SYN corresponde ao início de uma tentativa de estabelecimento de ligação TCP. A resposta RST, ACK indica que o destino rejeitou a ligação, por não existir nenhum serviço a escutar na porta indicada.

Este comportamento confirma o funcionamento do protocolo TCP e foi corretamente identificado pelo sniffer.

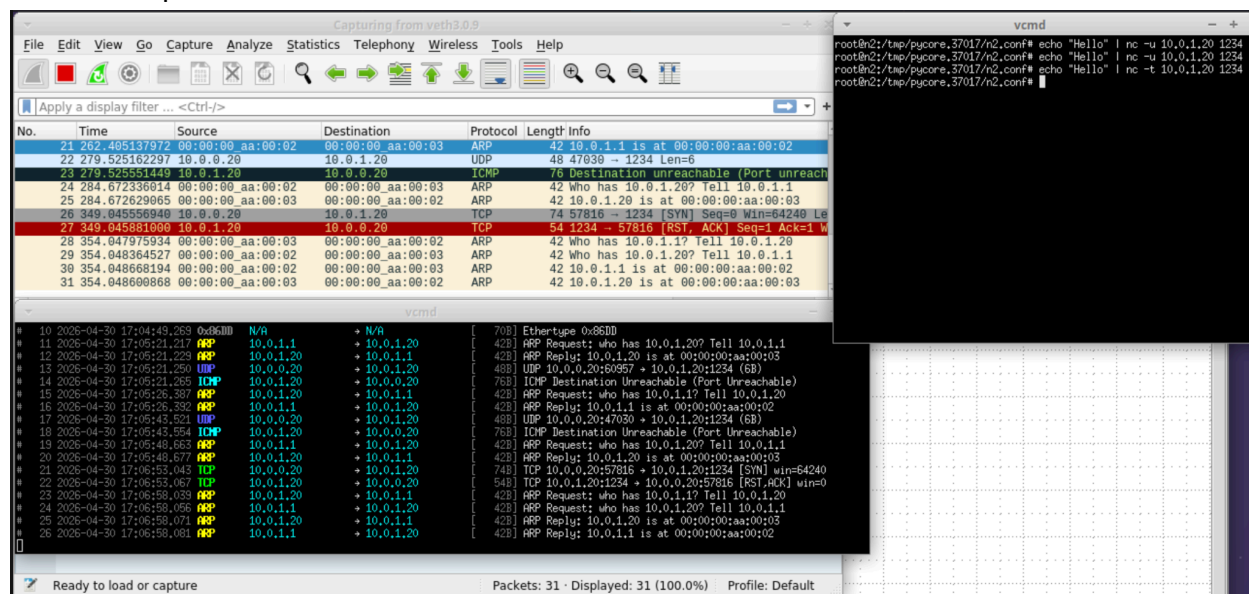


Figura 4 — Captura de tráfego no CORE evidenciando tentativa de ligação TCP (SYN) e respectiva resposta RST/ACK, bem como comunicação ARP, UDP e mensagens ICMP associadas.

8.7 Validação com Wireshark

Os resultados obtidos foram comparados com os capturados no Wireshark, ferramenta amplamente utilizada para análise de tráfego de rede.

A comparação permitiu verificar que:

- os protocolos identificados coincidem (ARP, UDP, ICMP, TCP);
- os endereços IP e MAC são consistentes;
- as sequências de comunicação observadas são equivalentes.

Esta validação confirma a correção da implementação do sniffer.

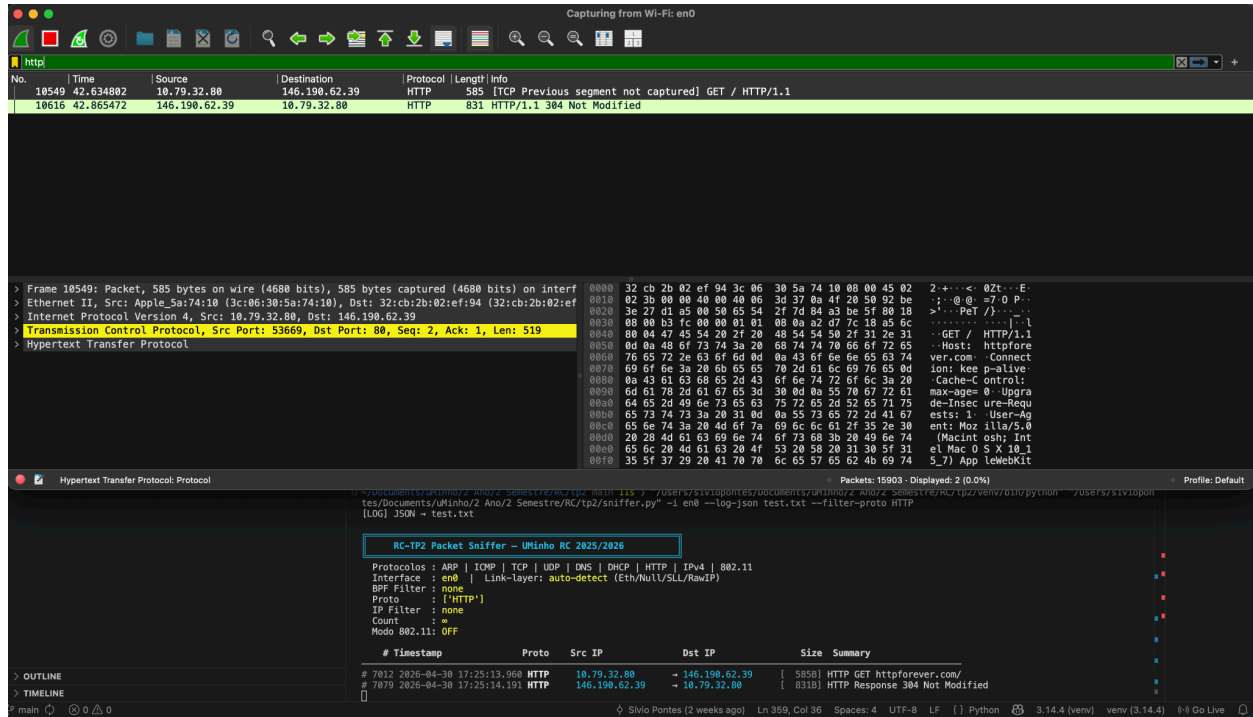


Figura 5 — Captura de tráfego HTTP evidenciando um pedido GET e a respectiva resposta HTTP/1.1 304 Not Modified, incluindo os detalhes do protocolo TCP e o conteúdo da mensagem.

8.8 Considerações Finais

Os testes realizados em ambiente CORE permitiram validar o comportamento do sniffer num cenário de rede controlado, com múltiplos nós e sub-redes.

Ao contrário dos testes em ambiente real, onde o tráfego é parcialmente imprevisível, o CORE permitiu gerar situações específicas, como erros de comunicação e ausência de serviços, possibilitando a observação de protocolos como ICMP (Destination Unreachable).

A combinação de ambos os ambientes de teste (real e emulado) permitiu uma validação mais completa e robusta da aplicação desenvolvida.

9. Limitações

Apesar do sniffer desenvolvido apresentar um conjunto alargado de funcionalidades e suportar múltiplos protocolos, existem algumas limitações inerentes à sua implementação e ao contexto de execução.

9.1 Limitações na Análise de Protocolos Aplicacionais

Uma das principais limitações do sistema está relacionada com a análise de protocolos ao nível da aplicação, nomeadamente no caso de comunicações cifradas.

Atualmente, o sniffer apenas consegue analisar tráfego HTTP não cifrado (portas 80 e 8080). No entanto, a maioria do tráfego web moderno utiliza HTTPS, que recorre a mecanismos de encriptação (TLS/SSL), impedindo a inspeção do conteúdo do payload.

Como consequência, embora seja possível identificar pacotes TCP associados a HTTPS, não é possível interpretar o conteúdo das mensagens ao nível da aplicação.

9.2 Dependência de Privilégios de Sistema

A captura de pacotes requer privilégios elevados do sistema operativo, sendo necessário executar o sniffer com permissões de administrador (por exemplo, através do comando sudo).

Esta dependência pode limitar a utilização da aplicação em ambientes onde o utilizador não possui permissões adequadas, reduzindo a portabilidade do sistema.

9.3 Dependência da Interface de Rede

O funcionamento do sniffer depende diretamente da interface de rede utilizada. Em sistemas com múltiplas interfaces (físicas, virtuais ou de túnel), a escolha incorreta da interface pode resultar na ausência de tráfego relevante.

Embora o sistema inclua mecanismos de listagem e seleção interativa de interfaces, esta dependência continua a ser um fator crítico para o correto funcionamento da aplicação.

9.4 Limitações na Captura de Tráfego Específico

Alguns protocolos, como o DHCP, não são facilmente observáveis durante testes normais, uma vez que apenas são utilizados em momentos específicos, como a inicialização da interface de rede ou a renovação do endereço IP.

Como resultado, apesar de o sniffer suportar a análise deste protocolo, nem sempre é possível validar o seu funcionamento em condições normais de utilização.

9.5 Suporte Limitado para Redes 802.11

O sistema inclui suporte para parsing de tramas IEEE 802.11 através do parâmetro `--dot11`. No entanto, a utilização desta funcionalidade depende do suporte da interface de rede e da ativação do modo monitor.

Na maioria dos sistemas, este modo não se encontra disponível por defeito, limitando a utilização prática desta funcionalidade.

10. Conclusão

O presente trabalho permitiu o desenvolvimento de um packet sniffer funcional, capaz de capturar, analisar e interpretar tráfego de rede em diferentes contextos. A aplicação implementada demonstrou ser capaz de identificar protocolos pertencentes a várias camadas da pilha de comunicação, incluindo ARP, ICMP, TCP, UDP, DNS, HTTP e DHCP.

A utilização da biblioteca Scapy para captura de pacotes, combinada com o parsing manual dos dados, possibilitou uma compreensão aprofundada da estrutura dos protocolos e do funcionamento interno das comunicações em rede. A implementação de mecanismos de filtragem e logging contribuiu para aumentar a flexibilidade e utilidade da aplicação.

Os testes realizados em ambiente real, utilizando a interface `en0` em macOS, permitiram validar o comportamento do sniffer em cenários reais, incluindo resolução de nomes DNS, comunicação HTTP e tráfego ICMP. A comparação com o Wireshark confirmou a correção dos resultados obtidos.

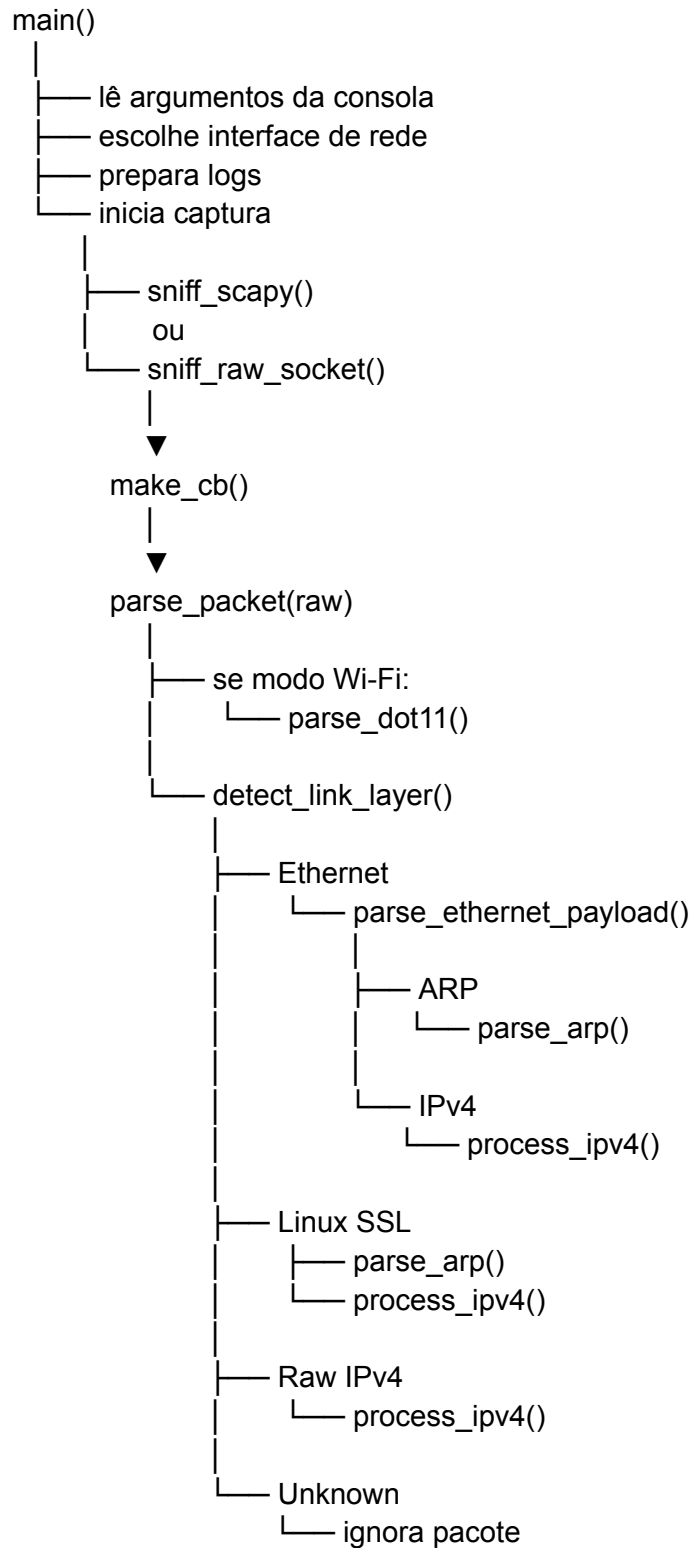
Por outro lado, a utilização do CORE possibilitou a criação de um ambiente controlado, onde foi possível gerar tráfego específico e observar comportamentos mais detalhados, como mensagens ARP, erros ICMP e tentativas de ligação TCP. Esta abordagem complementou os testes em ambiente real, proporcionando uma validação mais completa do sistema.

Apesar das limitações identificadas, nomeadamente na análise de tráfego cifrado e na dependência de permissões do sistema, a aplicação cumpre os objetivos propostos no enunciado, demonstrando a capacidade de captura e análise de tráfego de rede.

Em termos de aprendizagem, este trabalho permitiu consolidar conhecimentos fundamentais sobre protocolos de rede, bem como desenvolver competências práticas na análise de tráfego e implementação de ferramentas de monitorização.

Como trabalho futuro, seria possível expandir o sistema para suportar novos protocolos, melhorar o desempenho da captura e integrar funcionalidades adicionais de análise automática de tráfego.

Resumo



Ethernet → começa com MAC destino, MAC origem e Ethertype

Raw IPv4 → começa diretamente pelo cabeçalho IP

Linux SSL → tem um cabeçalho especial de 16 bytes

SSL:

bytes 0–1 → packet type

bytes 2–3 → ARPHRD type

bytes 4–5 → tamanho do endereço

bytes 6–13 → endereço

bytes 14–15 → protocolo / Ethertype

bytes 16+ → payload

ETHERTYPE:

0x0800 → IPv4

0x0806 → ARP

0x86DD → IPv6

0x8100 → VLAN

DOT11:

tipo da frame

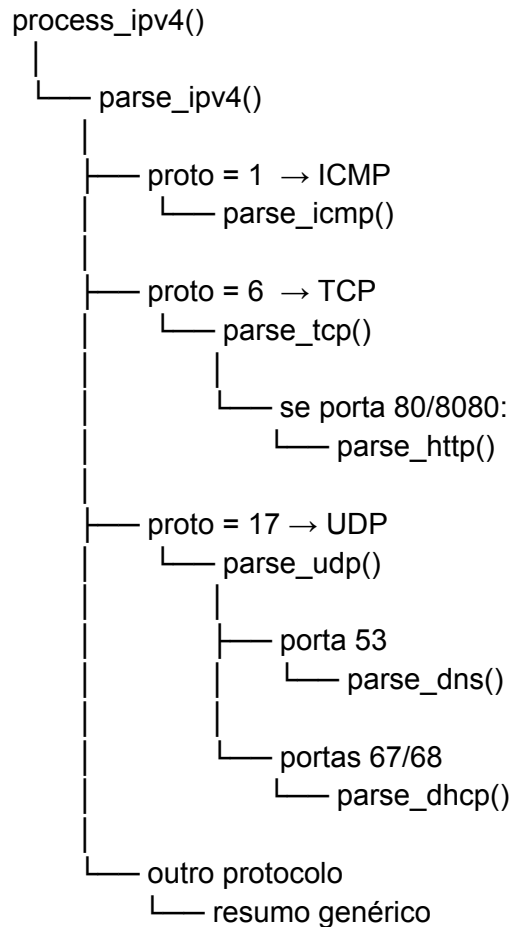
subtipo

BSSID

SSID

MAC origem

MAC destino



IMPORTS

argparse	Ler argumentos da linha de comandos
csv	Criar logs CSV
laddress	Converter IPv6 em texto, apenas para o parse dns
json	Criar logs JSON
socket	Capturar pacotes com raw socket
struct	Interpretar bytes crus parsers
sys	Terminar o programa erros
time	Medir duração e tempo relativo
datetime	Criar timestamp legíve
scapy.all.sniff	Capturar pacotes com Scapy
scapy.all.get_if_list	